

TerraZero: Procedural Driving Simulation for Zero-Demonstration Self-Play at Scale

Zhouchonghao Wu^{1,*}, Akshay Rangesh^{1,*}, Weixin Li¹, Wei-Jer Chang^{1,2}, Zachary Lee¹, Tim Wang¹, Wei Zhan^{1,2,†}

¹Applied Intuition ²University of California, Berkeley

*Equal contribution. †Corresponding author: wei.zhan@applied.co

Project website: <https://terra-applied.github.io/TerraZero>

Abstract

Training robust autonomous driving agents requires a simulator that is fast enough for reinforcement learning at scale, realistic enough to ground behavior in real-world map structure, and diverse enough to cover the safety-critical long tail that logged driving data rarely contains. We present **TerraZero**, a procedural driving simulator and self-play training stack that meets these goals. A configurable C engine runs simulation on the CPU and policy inference on the GPU over a zero-copy data path, sustaining 1.3M agent-steps per second on a single server-grade GPU, significantly faster than existing object-level simulators, while retaining fidelity that lighter single-agent systems omit: heterogeneous agents, multiple dynamics models, and full traffic-rule enforcement. Rather than treating logged data as the training distribution, TerraZero uses it only as a source of real-world map geometry: it populates each map with a randomized cast of rule-based road users and signal controllers, and randomizes agent dynamics, rewards, and sizes per episode, so a single map yields an effectively unbounded supply of training scenarios. TerraZero trains every reported policy from scratch by reinforcement learning alone, with *zero* human demonstrations. It uses no imitation, no logged trajectories, and no fallback planner at inference, on a compute-efficient self-play recipe scaled across GPUs. The resulting policies generalize *zero-shot* across cities and datasets, including emergent left-hand-traffic driving without explicit supervision. As an ego policy, it is, to our knowledge, the first fully learned policy to top the InterPlan long-tail benchmark, ahead of much larger learned planners. On the routine-driving val14 benchmark it ranks among the best-performing approaches and is the safest, posting the best collision and time-to-collision scores. On Waymo Open Sim Agents realism the same recipe yields a sim agent that outperforms other demonstration-free methods and is competitive with the strongest reference-anchored self-play method. A single stack therefore serves both roles: it trains state-of-the-art demonstration-free driving policies across vehicle dynamics suited to different platforms such as cars and trucks, and sim agents that jointly control the full traffic set of vehicles, pedestrians, and cyclists, all with configurability broad enough to meet a wide range of needs.

CONTENTS

Abstract	1
Contents	2
1 Introduction	4
2 Related Work	6
2.1 Object-Level Driving Simulators	6
2.2 Traffic Simulation via Imitation Learning	7
2.3 Self-Play for Driving	7
3 Fast, Feature-Rich Traffic Simulation	7
3.1 Designing for Performance	8
3.2 Composable Simulation Substrate	9
3.3 Traffic Rule Enforcement	9
3.4 Data Pipeline	10
4 Procedural Scenario Generation	10
4.1 Scene Initialization	10
4.2 Goal Assignment	10
4.3 Actor Diversification	11
4.4 Rule-Based Non-Player Characters	11
4.5 Signal Control	12
5 A Robust Training Recipe	12
5.1 Compute-Efficient RL	12
5.2 Distributed Training	14
6 Experiments & Results	14
6.1 Simulation Throughput	14
6.2 Policy Training Setup	14
6.3 Planner Benchmarks	15
6.4 Sim Agent Benchmark	18
6.5 Generalization	21
6.6 Qualitative Analysis	23
7 Conclusion	23
References	24
A Simulation Constants	28
B Reward Function Details	28
C Observation Space Details	29
D Agent Initialization	31
D.1 Random Agent Placement	32

D.2	Behavioral Role Assignment	32
D.3	Hybrid Initialization	32
D.4	Kinematic Parameter Randomization	32
D.5	Goal Assignment	33
E	Training Configuration	33

1 Introduction

Real-world driving data is dominated by routine driving. Most logged miles consist of steady-state lane following, gentle curves, and routine stops, the scenarios an autonomous agent has least to gain from practicing. The long tail of safety-critical situations that largely determine deployment readiness (dense highway merges, aggressive cut-ins, near-miss pedestrian interactions, unprotected left turns through oncoming traffic) is rare, and therefore costly to cover, in real-world driving datasets. Imitation learning on logged trajectories struggles to teach behaviors that the logs barely contain, and collecting more data at fleet scale closes this distributional gap only slowly. Simulation offers a complementary path: a simulator can systematically manufacture the hard scenarios that matter most [18, 27, 46].

Existing simulation approaches occupy different points in a throughput–fidelity trade-off, and none fully resolves it. Object-level simulators built for speed, such as PufferDrive [10, 11], reach high throughput but support only a single agent type with fixed dynamics and no traffic signals. Feature-rich environments like SMARTS [57] and CARLA [18] offer the complexity needed for realistic evaluation but are orders of magnitude too slow for from-scratch reinforcement learning (RL) training at scale, where billions of environment steps are routine. A separate line of work builds learned traffic models (SimNet [3], BITS [51], Trajenglish [34]) that generate realistic trajectories for surrounding agents but produce largely non-reactive traffic, limiting their utility for closed-loop policy improvement. Self-play approaches like Gigaflow [13] show that reactive multi-agent training can yield robust driving policies, but they train on a small family of synthetic maps under a single dynamics model shared by every agent class, and they rely on randomization alone, rather than constructed scenario content, for exposure to the long tail.

TerraZero closes this gap through *procedural scenario generation*. It treats logged data as a source of real-world map geometry, and composes a stack of randomization mechanisms to manufacture hard scenes from each map, while a configurable C simulation engine supplies the throughput that large-scale RL demands. Each map is populated with a procedurally generated cast of rule-based road users (parked and reactive vehicles, planner-driven traffic, crashed-vehicle clusters, construction zones, and crossing and jaywalking pedestrians), driven through randomized traffic-signal controllers, with per-episode randomization of agent dynamics [43], reward weights, and bounding-box sizes layered on top. Because every axis varies from episode to episode, one map seeds a vast and non-repeating space of training scenarios. The maps themselves span three datasets: the Waymo Open Motion Dataset (WOMD) [20], nuPlan [5] across four cities that span both right- and left-hand traffic, and five synthetic CARLA [18] towns, giving broad coverage of intersection layouts, road topologies, and driving conventions. TerraZero provides a complete training stack from scenario loading through distributed Proximal Policy Optimization (PPO) and evaluation. Figure 1 summarizes the configuration surface and the training stack.

Our contributions are threefold:

- (1) A **fast, feature-rich object-level traffic simulator** for RL training. A configurable C engine runs simulation on the CPU and policy inference on the GPU, connected by a zero-copy data path with dense feature packing, 16-bit observations, non-uniform memory access (NUMA)-aware orchestration, and a compact binary scenario format. It supports heterogeneous agents (vehicles, pedestrians, cyclists), multiple dynamics models, and full traffic-rule enforcement, sustaining up to 2.8M agent-steps per second (SPS) on an 8-GPU node, to our knowledge significantly faster than any existing driving simulator, while retaining a fidelity that lighter single-agent systems omit.

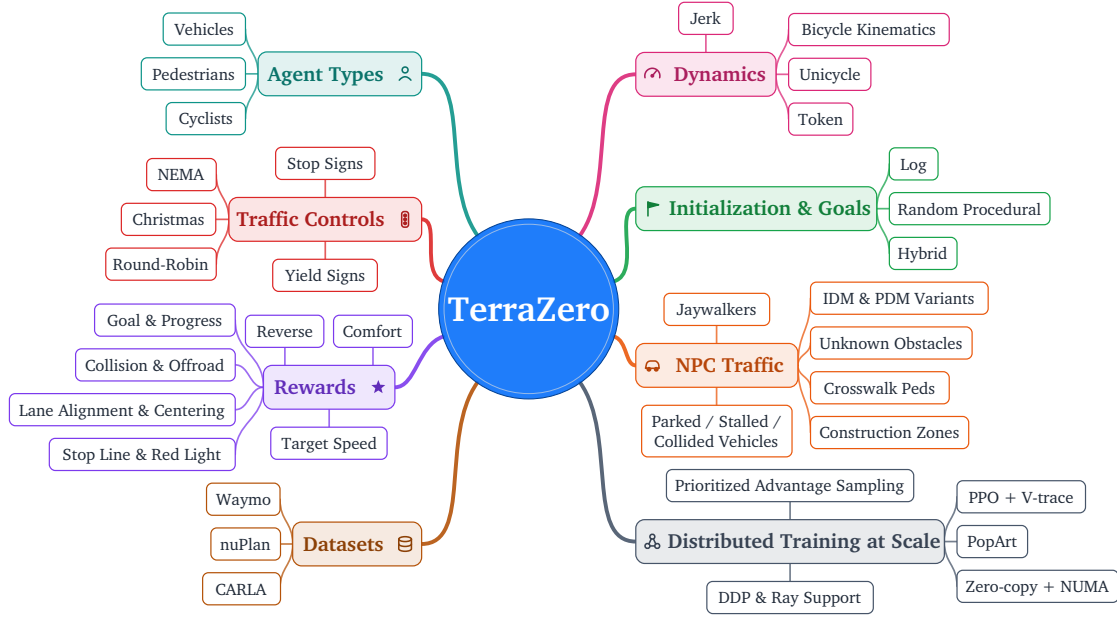


Fig. 1. TerraZero at a glance. TerraZero couples three pillars: a fast object-level simulator, a procedural scenario generator, and a self-play training recipe. Each branch is an axis the system can control: how agents move, who shares the road, how episodes begin, the rule-based traffic and signal controllers that populate the scene, the reward terms, the source datasets, and the machinery that scales training across GPUs. A configuration fixes some of these choices and randomizes the rest across episodes, which is how TerraZero manufactures hard scenarios from real-world map data.

- (2) A **procedural scenario generator** that treats logged data as the starting distribution rather than the training distribution. From real-world maps it manufactures diverse scenarios through randomized initialization and goal sampling, agent density and dimension randomization, a configurable population of rule-based road users, and three traffic-signal controllers, so a single map yields an effectively unbounded supply of training scenarios.
- (3) A **robust self-play training recipe** for object-level driving policies. Because the simulator is fast, the recipe trades sample efficiency for compute efficiency via saliency-prioritized sampling, and combines a compact feed-forward policy with V-trace off-policy corrections, PopArt value normalization, reward and kinematic domain randomization, and population play that breaks self-play symmetry, scaled across GPUs with synchronized normalization statistics. The *zero* in TerraZero names the training stance: zero human demonstrations. Because the recipe learns from scratch by reinforcement signal alone, it needs no imitation loss and no logged trajectories, and every result we report is trained this way. Log and hybrid initialization from logged data stay available as options that our reported policies do not use. The resulting policies generalize zero-shot across cities and datasets, including emergent left-hand-traffic driving without explicit supervision.

We validate these contributions on three public datasets (Waymo, nuPlan, CARLA), measuring driving performance on nuPlan val14 and InterPlan and sim-agent realism on the Waymo Open Sim Agents Challenge (WOSAC). Across the three, one recipe tops the InterPlan long-tail benchmark, ranks among the best-performing approaches on the routine-driving val14 benchmark and is the safest there by collision and time-to-collision, and outperforms other demonstration-free methods on WOSAC realism, evidence that one stack can serve both traffic simulation and high-performance planning.

Table 1. Comparison of object-level driving simulators. TerraZero combines C-level simulation speed with scenario fidelity (heterogeneous agents, multiple dynamics models, traffic signals) typically found only in slower, more feature-rich systems. Agent SPS is reported separately for a single consumer-grade GPU, a single server-grade GPU, and an 8× server-grade GPU node. Numbers marked [†] are figures reported by the original authors; all other SPS values are benchmarked under our setup. “Multi-node” indicates whether the system supports distributed training across multiple machines.

System	Backend	Agent SPS				Agent Types	Dyn. Models	Signals	Reactive	Data Sources
		Consumer	Server	8× Server	Multi-node					
Nocturne	C++	2K [†]	—	—	—	Vehicles	1	—	—	Waymo
SMARTS	Python	—	—	—	✓	Veh, Ped, Cyc	Multiple	✓	Partial	Synthetic (SUMO)
Waymax	JAX	—	319K [†]	—	✓	Veh, Ped, Cyc	2	✓	✓	Waymo
GPUDrive	Madrona	11K	22K	—	—	Vehicles	1	—	—	Waymo
PufferDrive	C	320K	745K	—	—	Vehicles	1	—	✓	Waymo
Gigaflow	—	—	—	530K [†]	—	Veh, Ped, Cyc	1	✓	✓	Synthetic (CARLA)
TerraZero	C	560K	1.3M	2.8M	✓	Veh, Ped, Cyc	Multiple	✓	✓	Waymo, nuPlan, CARLA

2 Related Work

2.1 Object-Level Driving Simulators

The choice of simulation abstraction sets the ceiling on reinforcement learning scale. Pixel-level simulators such as CARLA [18] render photorealistic urban scenes with full sensor suites, but their rendering cost caps throughput at tens of frames per second. The long-tail situations that most determine deployment readiness are hard to learn, and learning them from scratch through self-play is harder still; both demand orders of magnitude more environment steps than pixel-level rendering can supply. *Object-level* simulators represent the world as structured state (positions, velocities, road geometry) rather than pixels, trading perceptual realism for the throughput that large-scale RL requires. TerraZero takes this trade.

Nocturne [46] introduced object-level driving simulation with a C++ backend over Waymo Open Motion Dataset scenarios, achieving large speedups over pixel-level alternatives. It remains narrow, however: it drives vehicles only, models no traffic signals, and offers a single dynamics model. SMARTS [57] is more feature-rich, with multiple agent types and flexible scenario specification, but its Python simulation loop bottlenecks on the CPU and caps throughput for large-scale RL. Hardware acceleration lifts that ceiling. Waymax [21] runs batched rollouts in JAX for planning, behavior prediction, and sim-agent research, and GPUDrive [27] lowers observation, reward, and dynamics functions to CUDA on the Madrona game engine, reporting roughly one million steps per second under heavily batched rollouts. Both stay narrow in scenario content: Waymax centers on the WOMB format and a benchmark-oriented API rather than a distributed self-play stack, while GPUDrive drives vehicles only with no signal modeling, so under a realistic configuration (Table 1) its sustained agent throughput falls well below that headline figure. PufferDrive [10, 11] is the closest antecedent, a fast C backend over Waymo scenarios, but it too supports a single agent type with limited dynamics.

Gigaflow [13] pairs a well-engineered platform with self-play to produce robust driving policies. As a training substrate it carries three limitations. First, it trains on a small family of synthetic CARLA-derived maps, affine variants of one fixed network rather than real-world geometry. Second, its scenes hold only policy-driven agents and static obstacles modeled as untyped immobile vehicles, leaving out the real-world situations that any road-ready driving policy must expect to encounter, such as construction zones, crashed-vehicle clusters, curb-parked cars, and jaywalking pedestrians, the long-tail cases that benchmarks such as InterPlan [23] probe. Third, it drives every agent class through a single bicycle model, covering trucks as size variants of the same box, and at evaluation hands pedestrians to a scripted controller in the style of the Intelligent Driver Model (IDM) [44], so its unified control of pedestrians runs only inside its own simulator. Table 1 summarizes these distinctions.

2.2 Traffic Simulation via Imitation Learning

A complementary line of work learns traffic agent behavior directly from logged driving data, producing simulated actors that reproduce the statistical properties of real traffic. SimNet [3] trains a neural network to generate reactive multi-agent rollouts from real-world observations. BITS [51] introduces a bi-level formulation that separates high-level route intentions from low-level trajectory generation. Trajenglish [34] reframes traffic modeling as next-token prediction over discretized trajectory tokens, leveraging the scalability of autoregressive sequence models. CTG++ [56] conditions traffic generation on natural language descriptions via diffusion models. TrafficBots [54] learns action-conditioned world models that can simulate multi-agent interactions. ITRA [39] formulates differentiable simulation that enables gradient-based optimization of traffic scenarios.

These methods excel at producing statistically faithful reproductions of logged traffic patterns and have shown strong results on realism benchmarks such as WOSAC [31]. Related planner and policy benchmarks such as NAVSIM [15] instead measure ego-driving performance under non-reactive pseudo-simulation. Their fidelity is bounded by the quality of the logs they learn from: WOMD, for instance, carries sparse traffic-signal annotations and very few cyclists, and a shift in sensor configuration or offboard-labeling accuracy can move the target they reproduce. These models are also optimized for fidelity to the *logged* distribution rather than for generating challenging, interactive scenarios, and most are either non-reactive to the ego agent’s decisions or offer only limited reactivity that degrades over long rollout horizons, which leaves them less suitable as training environments for RL.

2.3 Self-Play for Driving

Reinforcement learning for autonomous driving has shown increasing promise as simulation throughput has improved. Early work demonstrated RL-trained agents in simplified settings [28], with later efforts carrying the paradigm into domains as varied as off-road terrain [50], and more recent efforts scaling to complex multi-agent scenarios on object-level simulators [27, 46]. A key challenge is generating sufficiently diverse and challenging training scenarios: logged data provides realistic but mundane distributions, while hand-crafted scenarios are labor-intensive and inevitably incomplete.

Self-play offers a solution. By training an agent against copies of itself or its prior checkpoints, self-play creates an automatic curriculum: as the policy improves, the traffic it trains against becomes proportionally more competent, generating progressively harder interactions without manual design. This principle has driven breakthroughs across domains, from Go [40] to Dota [4] to multi-agent locomotion [2].

Gigaflow [13] gives the most comprehensive demonstration for driving, combining multi-agent self-play with domain randomization to produce policies robust to distributional shift, though it does so entirely on synthetic maps with homogeneous agent dynamics, as discussed above. SPACeR [7] stabilizes self-play with centralized reference models that anchor agent behavior against the policy drift of unconstrained co-adaptation, at the cost of maintaining a separate reference model. HR-PPO [12] regularizes self-play with imitation losses to keep policies human-like, which tethers behavior to the logged demonstration distribution rather than letting self-play explore freely beyond it. BehaviorBenchmark [16] contributes a standardized suite for the closed-loop behavior of learned driving policies; we cite its reported InterPlan RL baselines as comparison points rather than running its suite ourselves.

3 Fast, Feature-Rich Traffic Simulation

TerraZero is built around two goals that are usually in tension: a simulator fast enough for reinforcement learning at scale, and one feature-rich enough to represent the long-tail situations a policy

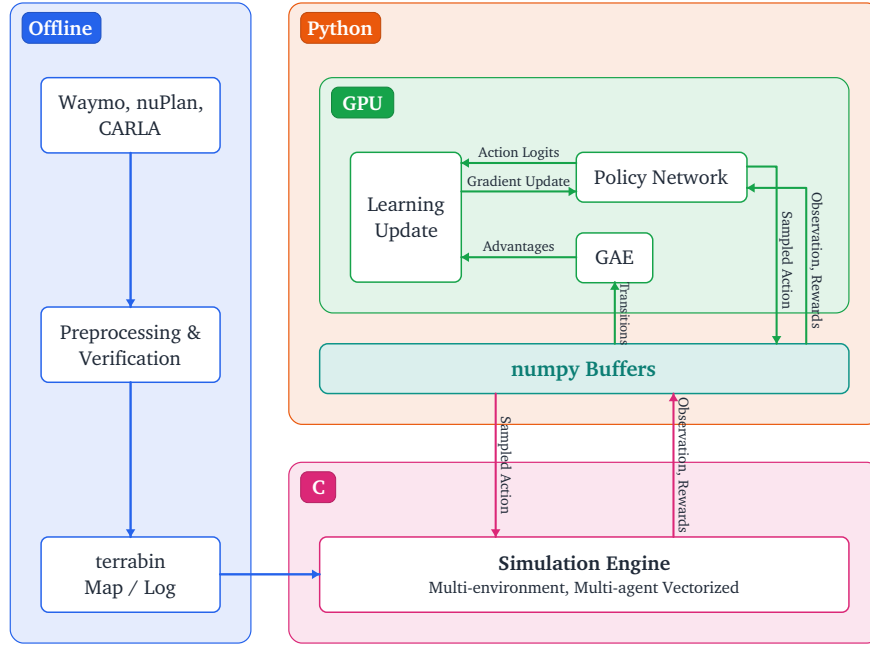


Fig. 2. System architecture. The offline data pipeline (left) converts Waymo, nuPlan, and CARLA scenarios into a compact binary scenario format (Section 3.4). At runtime, the C simulation engine (bottom) exchanges actions and observations with the GPU training loop (top) through shared, zero-copy numpy buffers, with the PPO learning update driving policy improvement.

must learn to handle. We achieve both by carefully partitioning work between the CPU and the GPU (Figure 2) and by implementing only the features that measurably improve training outcomes. Appendix A provides a reference table of simulation constants.

3.1 Designing for Performance

CPU/GPU division of labor. The object-level dynamics, observation construction, reward computation, and traffic-light logic run entirely in C, compiled into a CPython extension, while policy inference and learning run in PyTorch on the GPU [1]; the whole stack is built on the PufferLib vectorization and training framework [41]. Environments execute in parallel worker processes over a shared set of memory buffers, and the trainer consumes the full batch in a single device transfer and batched forward pass. This split keeps the CPU saturated with cheap, branchy simulation work and the GPU saturated with dense tensor math.

Low-overhead data path. The interface between the two is fully zero-copy: the C engine writes simulation state directly into the buffers that the training loop reads back as PyTorch tensors, eliminating the per-step serialization and allocation that dominate CPU-bound simulators, while host-to-device transfers overlap with kernel launches. TerraZero further sizes its buffers to the controlled agents actually present rather than padding to a fixed count, emits observations in 16-bit precision that the GPU reinterprets bit-for-bit to halve observation bandwidth, and pins each rank’s workers to the CPU cores local to its GPU’s NUMA node.

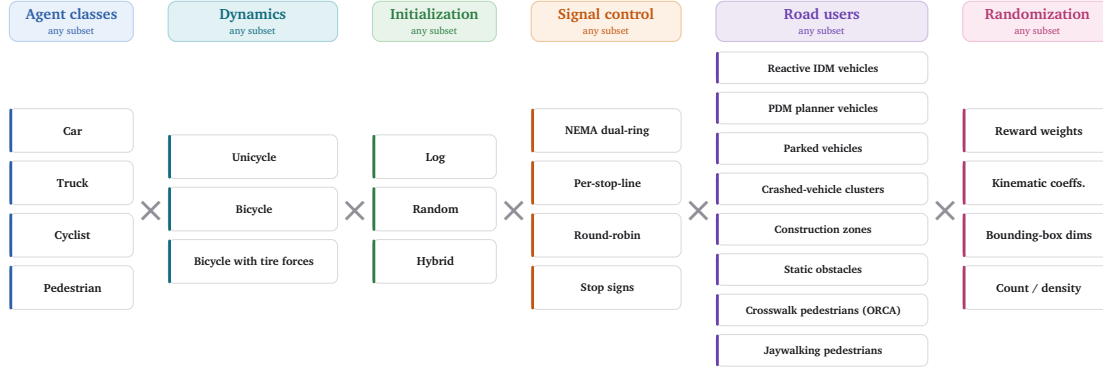


Fig. 3. Composable configuration axes. A scenario is composed by choosing a subset of options along each of six independent axes: agent classes, per-class dynamics, initialization, signal control, rule-based road users, and per-episode randomization. Because the axes compose freely, a single map seeds a combinatorially large space of scenarios.

3.2 Composable Simulation Substrate

Every subsystem in the engine is independently configurable, and the options compose. A scenario is assembled by choosing a subset of options along each axis in Figure 3: agent classes, per-class dynamics, initialization, signal control, rule-based road users, and per-episode randomization. The axes compose independently, so the reachable scenario space grows combinatorially in the per-axis choices; the road-user axis, for instance, admits any subset of its eight generators (Section 4.4). A new combination is a configuration change rather than a code change, which is what lets a single map seed the scenario distributions of Section 4. The observation, reward, and kinematic subsystems expose the same switchable control, and the policy network resizes automatically to the enabled feature set (Section 5).

Per-class dynamics. Each agent class is simulated under its own dynamics model: pedestrians under a unicycle model, cyclists and cars under a bicycle model, and trucks under a bicycle model augmented with tire cornering forces. Cars are actuated at the jerk level for smooth, jerk-limited control, while the other classes are actuated by acceleration. A single shared policy controls all classes through per-type action masking (Section 5).

3.3 Traffic Rule Enforcement

TerraZero enforces the principal driving rules directly at the simulation layer: each violation drives a reward penalty (Section 5), enters the evaluation metrics, and can trigger a configurable consequence such as stopping or removing the offending agent.

Collisions and off-road. Collisions between agents and between an agent and an obstacle are resolved by a separating-axis test over oriented bounding boxes. Off-road excursions are caught by testing the agent’s footprint against road-edge and sidewalk boundaries, with an elevation gate so that overpasses are ignored.

Lane direction. Lane-direction compliance is measured from the heading residual between the agent and its current lane, so reversed travel is penalized but never masked. Map handedness (left- versus right-hand drive) is fixed per map and carried in the lane geometry rather than in a separate check, so the same rule logic applies on both sides of the road.

Signals and stop signs. Signalized and sign-controlled intersections share a common geometric test: the agent’s footprint is checked against per-stop-line regions, and entry and exit are tracked as an edge-triggered passage. A red-light violation registers when the agent’s swept path crosses the stop bar on a red tick, not merely for waiting within the region on red, so a vehicle that halts at the line and proceeds on green is not penalized. Stop signs require the agent to hold below a speed threshold for a sampled dwell time before the line clears. Pedestrians are exempt from intersection rules, and cyclists are treated as vehicles. The signal state these checks consult comes from the controllers of Section 4.5.

3.4 Data Pipeline

An offline pipeline (the left branch of Figure 2) converts each dataset into a single common representation (inspired by ScenarioNet [30]), decoupling source-specific parsing from the simulator. Separate converters ingest Waymo, nuPlan, and CARLA, and a builder compiles the shared representation into simulation-ready geometry: it identifies intersections, ties stop lines and traffic signals to their lanes, and resolves legal travel directions and speed limits. Intersections come from a source’s own junction geometry where it provides one and are otherwise inferred from lane topology.

The builder validates each scene and emits it as a compact binary *terrabin* file that the engine reads directly. This keeps the simulator independent of any single data source: supporting a new dataset requires only a new converter. At training time, scenarios are streamed and sampled so that each batch stays balanced across regions and scenario types without holding the full dataset in memory.

4 Procedural Scenario Generation

A single real-world map is the seed for thousands of distinct training scenarios. TerraZero procedurally generates a superset of the logged data rather than being bounded by it: recorded map geometry and trajectories anchor where roads run and where agents may begin, and a stack of composable randomization mechanisms expands each seed into the far broader distribution the policy actually trains on.

4.1 Scene Initialization

TerraZero supports three initialization modes: log, random, and hybrid. In log mode, agents are instantiated directly from logged dataset trajectories and inherit their recorded goals. In random mode, the mode used for our default training runs, agents are procedurally placed per class under rejection sampling, so that every agent starts collision-free and compliant with the traffic rules, on a lane from which a sufficiently distant goal is reachable (Appendix D.1). The agent count is itself sampled from a configurable range whose upper bound may exceed the logged agent count, so scenes can be packed denser than any naturalistic recording. The hybrid mode mixes log and random at the granularity of whole environments through an independent per-environment draw, yielding a tunable blend across the batch rather than a within-scene mixture (Appendix D.3).

4.2 Goal Assignment

Procedurally initialized agents need navigation goals, which are sampled by a forward walk over the lane-topology graph up to a bounded arc length, filtered so that the goal lies ahead of the agent (Appendix D.5). A configurable goal-dropout probability periodically hides the goal to encourage robust, non-goal-reliant behavior. When an agent reaches its goal, it either receives a freshly sampled goal further along the topology or halts in place, according to configuration.

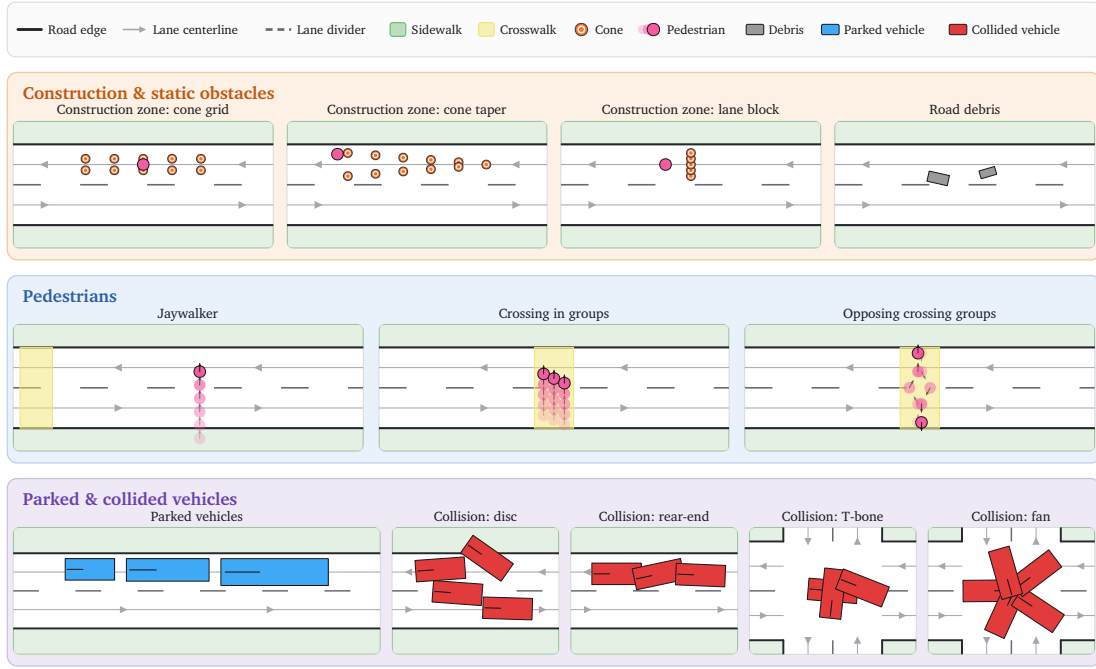


Fig. 4. Rule-based road users. Schematic of how TerraZero procedurally generates its scripted road users, drawn on synthetic whole-lane frames with solid road edges, white dashed lane dividers, grey direction-carrying centerlines, and green sidewalks. The top row shows the construction and static-obstacle generators that close a lane (a tiled cone grid, a diagonal taper, and a perpendicular lane-block, each with a stationary worker, alongside random road debris); the middle row the pedestrian generators as faded temporal trails (a mid-block jaywalker, a one-direction pack, and an opposing bidirectional group bowing apart under ORCA avoidance); and the bottom row the vehicle generators (curb-anchored parked cars, buses, and trucks, and the four pre-arranged collided-vehicle clusters: uniform disc, rear-end chain, T-bone, and outward fan). The legend keys the road-surface markings and the entity glyphs.

4.3 Actor Diversification

Beyond placement, the actors themselves are diversified. Each agent’s bounding-box dimensions are sampled per episode from type-specific ranges, with vehicles further split across car, truck, and bus size classes (Appendix D.1). Sampling dimensions independently of the source recording exposes the policy to a continuum of footprints, from compact cars to long buses and trucks, rather than the fixed sizes of any single dataset.

4.4 Rule-Based Non-Player Characters

Alongside the learned agents, TerraZero populates scenes with rule-based road users, or non-player characters (NPCs), which enrich interaction and, in self-play, help break the symmetry of a single shared policy training against copies of itself. The system offers several classes of these characters, and a scene draws a configurable mix of them that varies from episode to episode; the mixed presets used in our default training compose all of the spawned subsystems in one scene. Figure 4 shows how these scripted road users are procedurally generated.

Reactive vehicles. Reactive vehicles are driven either by an IDM [44] longitudinal controller paired with pure-pursuit steering, or by a closed-loop planner in the style of PDM-Closed [14] that forward-simulates a grid of lateral-offset $\times$ IDM-parameter proposals and selects the trajectory maximizing

a weighted score of progress, time-to-collision, comfort, and lane-keeping. IDM vehicles sample among default, assertive, and cautious behavior modes that rescale the IDM gains, and may reroll that mode within an episode, producing heterogeneous and time-varying driving styles.

Static actors. Static actors stay fixed for the episode once placed. They include parked vehicles along curbs spanning the car, bus, and truck size classes, pre-arranged crashed-vehicle clusters in a uniform-disc, rear-end chain, T-bone, or outward-fan layout, construction zones built from tiled traffic cones in grid, taper, or lane-block closures with optional stationary workers, and isolated static obstacles offset from the lane center.

Pedestrians and cyclists. Pedestrians cross at crosswalks under a reciprocal collision-avoidance controller (ORCA), arriving on a Poisson schedule sampled per crosswalk as single crossers, packs, bidirectional flows, or staggered sequences, and also jaywalk mid-block away from marked crossings, walking perpendicular to the road until they clear the far edge. Cyclists have no scripted controller; TerraZero populates them as NPCs only through log replay, though the learned policy still controls them as agents.

4.5 Signal Control

The intersections in a scene are driven by one of three configurable traffic-light controllers; detection of the resulting violations is handled by the engine and described in Section 3.3. A *NEMA* controller runs the standard dual-ring, eight-phase concurrency plan [32]: each ring advances its phases through green, yellow, and all-red intervals, and the two rings cross barriers together so that conflicting movements never run at once. The *Christmas controller*, used in our default training composition, instead cycles each stop line independently through red, green, and yellow with dwell times drawn from a LogNormal distribution, so signals across a map stay uncoordinated. A *round-robin* controller is the simplest: one approach leg holds green at a time, and the green window rotates leg by leg in canonical NEMA order. Two further mechanisms inject per-scene variety: the initial phase of each intersection is randomized at reset, and protected left turns are flipped to permissive by an independent per-intersection draw, so the same map presents different signal timing and turn permissions across episodes.

5 A Robust Training Recipe

TerraZero is trained with a self-play PPO recipe [38] built on PufferLib [41] and deliberately co-designed with the fast object-level simulator of Section 3. Training is *tabula rasa*: the policy starts from randomly initialized weights and learns from reinforcement signal alone, with zero human demonstrations: no imitation loss and no dependence on logged trajectories. Logged data enters only as map geometry and as the optional starting distribution of Section 4, so the default composition initializes agents randomly, and log and hybrid initialization are available when logged initial states are preferred but never required. The central observation behind the recipe is that when roll-outs are cheap, the right currency is *compute* efficiency rather than sample efficiency: it is worth discarding low-value samples and spending the freed compute on aggressive off-policy corrections, value normalization, and a heterogeneous agent population.

5.1 Compute-Efficient RL

Saliency-prioritized sampling. Rather than sweeping every collected transition, TerraZero draws a fixed number of minibatches by sampling trajectory segments *with replacement* in proportion to their priority $p_i \propto (\sum_t |\hat{A}_{i,t}|)^\alpha$, following the prioritized-replay principle [35]. Because the simulator is fast, this is a favorable trade: high-advantage segments are oversampled while near-zero-advantage

segments are simply skipped, yielding more gradient signal per unit of compute at the cost of re-visiting fewer unique samples. To correct the induced bias we apply importance-sampling weights $w_i = (N p_i)^{-\beta}$ with β annealed upward over training.

Compact feed-forward policy. The default TerraZero policy is a compact multilayer perceptron (MLP) network of approximately 3.5M parameters, in the style of Cusumano-Towner et al. [13] but with a simpler observation set: a single road-geometry resolution rather than separate coarse and fine map views, a single relative-goal target rather than a routed distance field with intermediate waypoints, and a smaller partner neighborhood (Appendix C). Per-group encoders (an ego MLP and permutation-invariant Deep Sets encoders [52] for the road, partner, and traffic sets) project each group to an embedding, which is concatenated and passed through a three-layer, 1024-unit MLP shared by the actor and value heads. The value bootstrap is handled separately and is preserved across truncations (next paragraph).

Markov decision process design. Each agent observes its ego state (speed, dimensions, heading, steering, acceleration, a relative goal, and a one-hot agent type), up to 20 partner agents within 50 m, and up to 200 nearby road segments, with the partner and road sets consumed by the Deep Sets encoders. Vehicles act in a discrete control space, in either acceleration or jerk. We train two kinds of policy on this substrate: a *planner* that controls vehicles alone, and a heterogeneous *sim agent* (Section 6.4) that drives all three classes through separate per-type action heads over their own dynamics models: jerk for vehicles, a unicycle grid for pedestrians, and a compact bicycle grid for cyclists (Appendix E). The reward follows the Gigaflow shaping [13], combining a goal bonus with collision, off-road, comfort, lane-alignment, lane-centering, velocity, and reverse-driving penalties; the full specification and default weights are given in Appendix B, and observation feature groups in Appendix C.

Value estimation and learning dynamics. Advantages are computed by generalized advantage estimation (GAE) [37], augmented with three stability mechanisms. *V-trace off-policy corrections* [19] clip the temporal-difference and trace terms; because the policy changes between rollout collection and the gradient update, the clipped importance ratio from each minibatch is stored and used to recompute advantages at the start of the next epoch. Setting both clips to infinity recovers standard GAE. True terminals zero the value bootstrap while truncations preserve a saved $V(s_{\text{final}})$ from the pre-set observation; both cut the λ trace, so the policy is never penalized for surviving near a scenario boundary, a documented bias in time-limited RL [33]. *PopArt value normalization* [45] addresses return magnitudes that vary by orders of magnitude across scenarios: running mean μ and standard deviation σ are updated by an exponential moving average each epoch, and the value head is analytically rescaled so that its denormalized predictions are preserved across statistic updates. Value targets are computed in normalized space while advantages enter GAE on the true return scale. The three mechanisms (priority sampling, V-trace, and PopArt) address complementary aspects of the off-policy self-play optimization; Appendix E lists the values of every optimization hyperparameter.

Domain randomization over rewards and dynamics. The training environment is randomized along two axes that the policy observes and must adapt to. The reward function is a vector of weighted terms (goal, collision, off-road, comfort, lane-alignment, lane-centering, velocity, and reverse-driving, among others), each of which can be disabled or randomized per agent within a range sampled at episode reset, following the heterogeneous-agent formulation of Gigaflow [13]. The same randomization applies to the four multiplicative kinematic-scaling coefficients (c_{throttle} , c_{steer} , c_{acc} , c_{vel}) that govern each agent’s acceleration, steering, and speed limits. Both the sampled reward weights and the kinematic coefficients enter the ego observation (reward- and kinematic-conditioned observations), so the policy adapts to the current regime online rather than memorizing a single one, and its behavior can be steered at inference by adjusting the weight vector without retraining (Appendices B, D.4, and C). Our default training composition randomizes both axes.

Goal dropout. *Goal dropout* keeps the policy from over-relying on its goal input. It marks a per-type fraction of the policy-controlled agents at each reset (0.3 of vehicles by default) and hides the goal of a marked agent by zeroing its relative-goal observation, while a separate visible flag tells the network the goal is masked rather than positioned at the origin. The marked agent must continue driving sensibly with its goal hidden, following the road structure until it leaves the map or the episode ends. TerraZero also exposes an optional observation-noise model, off by default, that adds clipped Gaussian perturbations and slot dropout to the ego, partner, and road features for perception-robustness experiments.

Breaking self-play asymmetry with population play. A single shared policy controls all learning agents, so self-play pits the policy against copies of itself; left unchecked this invites degenerate, perfectly symmetric equilibria. TerraZero breaks the symmetry by populating each scene with a heterogeneous *population* of controllers rather than a league of frozen checkpoints. Two ingredients do the work. First, the per-agent kinematic and reward domain randomization described above makes co-trained policy agents behave heterogeneously even under one network. Second, scenes are populated with the rule-based road users of Section 4.4: reactive IDM controllers [44] and closed-loop PDM planners [14], alongside parked vehicles, crashed-vehicle clusters, construction zones, and crossing or jaywalking pedestrians, in a configurable mix that varies from scene to scene (Figure 4). The learned policy must therefore remain robust to interacting with agents whose behavior it does not control, which is precisely the situation it faces in mixed-control deployment and evaluation.

5.2 Distributed Training

TerraZero scales across GPUs and nodes by data parallelism: each rank runs its own simulation environments and computes local gradients over its own rollouts, and the gradients are all-reduced across ranks into a single global mean update.

The subtlety in distributed self-play is not the gradient all-reduce but keeping the recipe’s *normalization statistics* globally consistent, since each stability mechanism maintains running statistics that are meaningless if computed per-rank. TerraZero synchronizes these across ranks so that advantage normalization, PopArt value rescaling, and joint priority sampling all operate on a single global scale rather than on any one rank’s shard.

6 Experiments & Results

6.1 Simulation Throughput

A key claim of TerraZero is high throughput *without* sacrificing scenario fidelity. Figure 5 shows agent-steps-per-second scaling across hardware, and the head-to-head comparison against existing simulators is summarized in Table 1 (Section 2). Measured on a representative training job, TerraZero sustains 560K agent-steps per second on a single consumer GPU, 1.3M on a single server-grade GPU, and 2.8M on an 8-GPU server node, to our knowledge significantly faster than any existing driving simulator. It reaches these rates while supporting three agent classes, full traffic-signal state machines, and reactive traffic, a fidelity that the single-agent backends it outpaces do not provide.

6.2 Policy Training Setup

Training data. We source driving scenarios from three datasets, each converted to the terrabin format of Section 3.4, and we match each dataset to the experiments it supports. nuPlan [5] supplies the map geometry for the planner evaluated on val14 (Section 6.3.1) and InterPlan (Section 6.3.2). WOMD [20] supplies the map geometry for the sim agent evaluated on WOSAC (Section 6.4), which

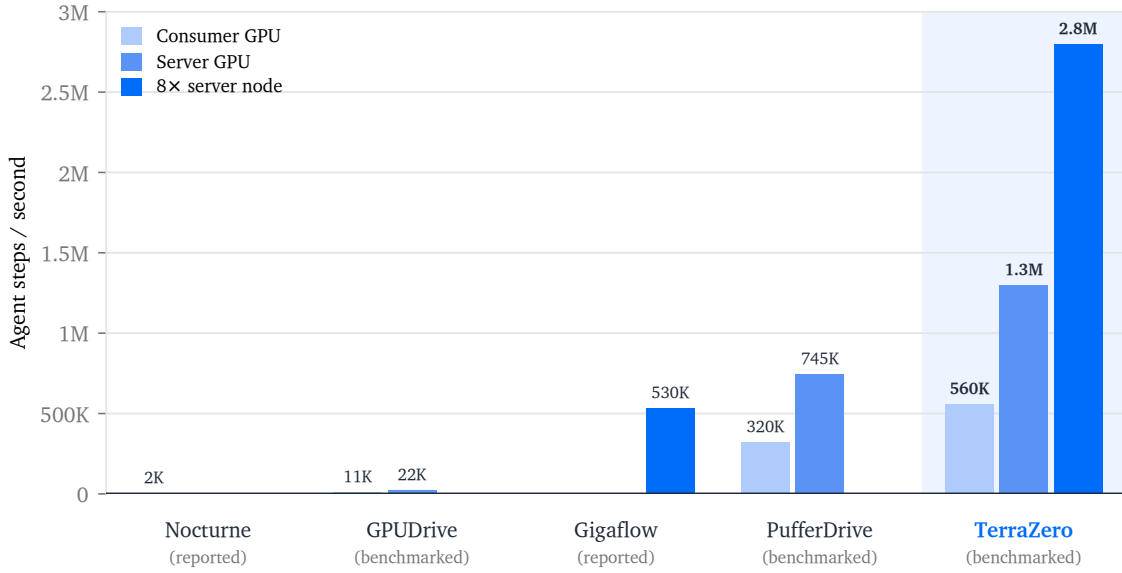


Fig. 5. Training throughput. Agent steps per second across hardware configurations. TerraZero achieves competitive throughput on a single consumer GPU and the highest throughput on server hardware, while retaining full scenario fidelity including traffic signals, heterogeneous agents, and reactive traffic.

trains on the full WOMB training split and is scored zero-shot on the WOSAC validation sets described there. The transfer study (Section 6.5) draws on all three sources, and their relative scale sets its context.

WOMB is the largest of the three: our direct converter yields about 576K scenarios, of which 487K form the training split. nuPlan contributes about 262K scenarios across its four cities, distributed unevenly: Las Vegas is by far the largest and Boston the smallest, with Pittsburgh and Singapore in between. CARLA [18] adds only five synthetic towns as map geometry. These sources span a broad distribution of intersections, highway merges, roundabouts, and urban corridors while differing in scale by orders of magnitude, from five hand-built CARLA maps to nearly half a million WOMB scenarios.

Policy and training. Unless otherwise noted, experiments use the compact multilayer-perceptron policy of Section 5 with jerk-based vehicle actions, trained from scratch with zero human demonstrations on nuPlan map geometry with PPO [38] and GAE [37], V-trace off-policy corrections [19], and PopArt value normalization [45], in bf16 mixed precision distributed across 16 NVIDIA A100 80GB GPUs. We apply kinematic, reward, and agent-density domain randomization throughout training, and the configuration of this planner checkpoint, shared by the val14 and InterPlan evaluations, is summarized in Appendix E. The heterogeneous sim agent of Section 6.4 is the exception: it uses separate per-type action heads, about 6.7M parameters, and trains on 32 GPUs.

6.3 Planner Benchmarks

We evaluate TerraZero as a *planner*, an ego driving policy, on two closed-loop nuPlan [5] benchmarks. The standard val14 split (Section 6.3.1), run through the PlanTF evaluation suite [9], measures the fat body of routine urban driving. The InterPlan long-tail suite (Section 6.3.2) [23] rewrites logged nuPlan scenes into the rare, out-of-distribution situations that val14 seldom contains, such as construction zones, accident sites, and jaywalking pedestrians. The two benchmarks score the policy with the official nuPlan closed-loop reactive score and execute its actions through the benchmark’s

Table 2. Driving policy evaluation on nuPlan val14 (closed-loop reactive). All metrics are higher-is-better. Type: Rule = rule-based; IL = imitation learning; Hybr. = hybrid rule+learned; RL = reinforcement learning; Replay = log replay with tracking controller. Asterisk (*) denotes author re-trained variants; dashes mark component scores the source paper does not report. SPDM appears at two proposal budgets N_p , its val14-best ($N_p=15$) and its InterPlan-best ($N_p=60$; Section 6.3.2). **Bold** indicates best; underline indicates second best. The Log Replay reference row is excluded from the best and second-best marking.

Planner	Type	Score $\uparrow$	Ego Prog. $\uparrow$	No AF-Coll. $\uparrow$	Comfort $\uparrow$	TTC $\uparrow$	Drv. Dir. $\uparrow$	Speed Lim. $\uparrow$	Drv. Area $\uparrow$	Making Prog. $\uparrow$
Log Replay (iLQR)	Replay	82.05	99.32	85.69	98.93	80.77	99.28	96.46	99.55	100.00
IDM [44]	Rule	77.33	85.20	89.22	93.11	81.57	99.24	97.20	93.02	96.33
PDM-Closed [14]	Rule	92.13	90.26	97.90	94.72	93.83	99.96	<u>99.83</u>	99.46	99.11
SPDM ($N_p=15$) [17]	Rule	92.28	—	—	—	—	—	—	—	—
SPDM ($N_p=60$) [17]	Rule	91.60	—	—	—	—	—	—	—	—
PLUTO [8]	Hybr.	89.66	86.11	97.09	91.86	<u>94.10</u>	<u>99.87</u>	98.86	99.28	98.57
FlowDrive w/ guidance + PDM scoring [47]	Hybr.	<u>92.96</u>	—	—	—	—	—	—	—	—
Urban Driver [36]	IL	53.05	87.17	72.45	99.28	66.37	96.87	81.66	83.36	94.28
PlanTF [9]	IL	76.14	77.21	95.21	93.56	90.61	99.33	98.50	96.33	89.53
Diffusion Planner* [55]	IL	82.73	85.88	93.07	89.45	88.19	99.82	98.29	97.85	96.42
Flow Planner [42]	IL	83.31	—	—	—	—	—	—	—	—
FlowDrive [47]	IL	85.37	—	—	—	—	—	—	—	—
Gigaflow [13]	RL	93.8	93.6	98.4	96.4	93.8	99.6	99.9	99.7	<u>99.0</u>
CaRL [26]	RL	90.60	<u>91.30</u>	97.05	88.55	92.31	99.15	99.36	99.91	99.11
TerraZero (Ours)	RL	92.27	88.91	99.11	<u>97.41</u>	96.06	99.24	99.45	99.28	97.85

standard nuPlan Linear-Quadratic Regulator (LQR) controller, the same low-level tracker every other planner runs under, so no controller-side change contributes to either score. At evaluation TerraZero conditions the policy for each benchmark through the reward- and kinematic-conditioned observations of Section 5, with no retraining, and aligns the eval-time goal and route sampling with the lane-level goals the policy saw during training (Section 4.2); the benchmarks’ own scoring stays unchanged.

6.3.1 nuPlan val14. Our primary ego-policy evaluation uses the nuPlan val14 closed-loop reactive benchmark [5], run through the PlanTF evaluation suite [9] and scored with the official nuPlan closed-loop reactive score. Each rollout runs the ego policy in closed loop against reactive traffic, and the score aggregates no-at-fault collision, time-to-collision (TTC), drivable-area, driving-direction, and speed-limit compliance, ego progress, making progress, and comfort into a single weighted value in $[0, 100]$, reported in Table 2 alongside its component terms. The comparison set spans the four planner families that populate the val14 leaderboard (rule-based, imitation, hybrid rule-plus-learned, and reinforcement learning), placing TerraZero against both the hand-engineered planners that top the benchmark and the learned policies nearest its own setting; the closest of these is Gigaflow [13], the one comparable large-scale GPU self-play system. TerraZero is, to our knowledge, the first policy trained exclusively via large-scale self-play to be competitive with dedicated nuPlan planners on val14.

TerraZero scores 92.27 on val14, second among the reinforcement-learning policies in the table, behind Gigaflow [13] and ahead of CaRL [26]. Apart from Gigaflow, the planners that outscore it are rule-based or hybrid: SPDM at its val14-tuned 15-proposal setting [17] and the guidance-plus-PDM variant of FlowDrive [47], which hand-craft trajectories or graft a rule-based scorer onto a learned model. That SPDM setting is the very one that gives back most of its long-tail performance on InterPlan (Section 6.3.2). TerraZero reaches this with a markedly leaner observation set than Gigaflow: a single road-geometry resolution rather than separate coarse and fine map views, a single relative goal rather than a routed distance field with intermediate waypoints, and a smaller partner neighborhood (Section 5). Because the policy depends on none of these engineered map features, it stays usable where the underlying lane and road topology is imperfect or broken, the regime that richer feature stacks handle poorly. The safety terms carry the score: TerraZero posts the best no-at-fault-collision (99.11) and time-to-collision (96.06) figures in the table, the safest of the compared

Table 3. Driving policy evaluation on the InterPlan long-tail benchmark, scored by the closed-loop reactive nuPlan score in $[0, 100]$ (higher is better). The InterPlan column reports the official 80-scenario split and the Full-Scale InterPlan column the larger 335-scenario set; dashes mark splits a method does not report. Type: Rule = rule-based; IL = imitation learning; Hybr. = hybrid, a learned or language model combined with a rule-based planner, including the LLM planners whose low-level controller is PDM-Closed; RL = reinforcement learning. Scores are from Hallgarten et al. [23], except Diffusion Planner, PPO, and PDM+PPO, which are from Distelzweig et al. [16] on the same reactive benchmark, and SPDM, FlowDrive, and Flow Planner, which are from their own papers. **Bold** is best and underline second best on the InterPlan split.

Planner	Type	InterPlan $\uparrow$	Full-Scale InterPlan $\uparrow$
IDM [44]	Rule	31	—
IDM+MOBIL [29, 44]	Rule	31	—
PDM-Closed [14]	Rule	42	—
SPDM ($N_p=15$) [17]	Rule	42.00	—
SPDM ($N_p=60$) [17]	Rule	<u>63.66</u>	—
DTPP [24]	Hybr.	25	—
HybridLLMPlanner (GPT-3.5) [23]	Hybr.	40	—
HybridLLMPlanner (Llama-13B) [23]	Hybr.	48	—
HybridLLMPlanner (Llama-7B) [23]	Hybr.	53	—
PDM+PPO [16]	Hybr.	43.2	—
FlowDrive w/ guidance + PDM scoring [47]	Hybr.	44.05	—
Urban Driver [36]	IL	4	—
GC-PGP [22]	IL	10	—
GameFormer [25]	IL	11	—
PDM-Open [14]	IL	25	—
Diffusion Planner [55]	IL	25.8	—
FlowDrive [47]	IL	36.96	—
Flow Planner [42]	IL	—	61.82
PPO [16]	RL	42.1	—
TerraZero (Ours)	RL	67.87	67.71

planners on both, and the second-best comfort, so the gap to the leaders sits in ego progress and making progress rather than in safety.

This result covers only the routine driving that dominates val14. Gigaflow trains on synthetic maps under a single standard-driving distribution and reports no long-tail benchmark, whereas TerraZero trains on both the fat body of routine driving and the procedurally generated long tail of Section 4.4. The next benchmark tests whether that long-tail training pays off where val14 is silent.

6.3.2 InterPlan. We next evaluate TerraZero on InterPlan [23], a benchmark that stress-tests planning in the long-tail, out-of-distribution situations that val14 rarely contains. InterPlan modifies logged nuPlan scenes into eight interactive scenario families: construction zones, accident sites, jay-walking pedestrians, nudging around a stopped vehicle, overtaking with oncoming traffic, and lane changes at low, medium, and high traffic density. Each planner drives in closed loop against reactive agents and is scored by the nuPlan closed-loop reactive score, which aggregates safety, progress, comfort, and compliance into a single value in $[0, 100]$. We report on the official 80-scenario split so that every method is scored on the same scenarios, and we report TerraZero on the larger 335-scenario set as a broader reference. The planner reported here and in Section 6.3.1 is a single checkpoint trained with one configuration, summarized in Appendix E.

TerraZero attains a score of 67.87 on the 80-scenario split, ahead of every prior planner; the closest is SPDM, a proposal-enriched PDM variant, at 63.66 [17], and the strongest LLM-based method, the 7B-parameter LLaMA HybridLLMPlanner, reaches 53 [23]. SPDM reaches 63.66 only at the

widest proposal budget it evaluates, 60 candidates per step, and its own sweep reveals a benchmark-specific tuning tax, which is why we tabulate its two endpoints as separate rows in Tables 2 and 3: the 15-proposal setting peaks on val14 at 92.28 but scores only 42.00 on InterPlan, while the 60-proposal setting that reaches 63.66 on InterPlan gives back val14, down to 91.60. No single SPDM configuration is strong on both benchmarks. The base PDM-Closed planner shows the same split, strong on val14 (92.13) yet weak on InterPlan (42), a tradeoff characteristic of rule-based planners, whose hand-designed proposal and scoring logic is tuned to one regime and does not carry to the long tail without cost elsewhere. A single TerraZero checkpoint essentially matches SPDM’s best val14 (92.27 versus 92.28) and exceeds its best InterPlan (67.87 versus 63.66), with no per-benchmark tuning. TerraZero reaches this with the compact multilayer-perceptron policy of Section 5, over three orders of magnitude smaller than a 7B-parameter language model, which points to procedural long-tail scenario generation rather than model scale as the source of the gain: the reported policy trains against a dense, procedurally generated population of construction cones, static obstacles, crashed-vehicle clusters, crossing and jaywalking pedestrians, and reactive IDM traffic (Section 4.4), so the long-tail families InterPlan probes resemble scenes the policy already practices.

The comparison also separates learned control from rule-based control. The strongest prior entries lean on a hand-crafted planner: the highest prior score is rule-based (SPDM at 63.66), and the competitive hybrids graft a rule-based planner or a language model onto a learned model, where HybridLLMPlanner pairs a language model with a PDM-Closed fallback, PDM+PPO [16] combines PDM with a learned policy, and FlowDrive climbs from 36.96 to 44.05 when its trajectories are reranked by the PDM scorer [47]. Purely learned planners that carry no such planner score far lower (Urban Driver 4, GameFormer 11, Diffusion Planner 25.8, FlowDrive 36.96, PPO 42.1). TerraZero uses no rule-based planner and no language model at inference, so the result comes from the learned policy alone. It is, to our knowledge, the first fully learned policy to reach the top of this long-tail benchmark.

The score is bounded by progress rather than by safety. On the 335-scenario set the route-progress term sits at 58.7 and time-to-collision at 91.3, while drivable-area compliance is near-perfect (99.1) and no-fault collision stays high (98.2). The policy keeps agents on the road and largely collision-free even in these rare, deliberately adversarial scenes, and the remaining headroom is in making forward progress through them. The same policy scores 67.71 on that set under the identical training configuration, ahead of Flow Planner at 61.82, the best prior learned planner reporting on it [42], which confirms that the result is not an artifact of the smaller official split.

6.4 Sim Agent Benchmark

We next evaluate TerraZero as a *sim agent*, a controllable traffic actor whose behavior must match the statistical distribution of real human driving. WOSAC characterizes each scenario as a 9.1 s WOMD recording at 10 Hz, uses the first 1.1 s as initial context, and scores how well a method reproduces the remaining 8 s for up to 128 agents under 32 stochastic closed-loop joint rollouts, combined into a realism meta-metric over kinematic, interactive, and map-based likelihood scores [31]. We report both editions on the Waymo validation set, evaluated zero-shot, and match the scenario selection of the competing methods in each edition: the 2023 edition uses the full validation split (the scenarios with at most 128 agents, the challenge cap), as in Gigaflow [13], while the 2024 edition uses the shared 880-scenario filtered validation subset used by CAT-K [53] and SPACeR [7], obtained by taking the 2% WOMD validation split after removing scenes with more than 128 agents. We use the 2023 edition to compare against Gigaflow, the closest demonstration-free self-play system to ours, and the 2024 edition to compare against more recent self-play methods.

Implementation details. The TerraZero sim agent is a heterogeneous self-play policy that controls vehicles, pedestrians, and cyclists jointly through a single shared network with separate per-type action

Table 4. WOSAC 2023 sim agent evaluation against demonstration-free self-play baselines, scored on the full Waymo validation split. All metrics are higher-is-better. TerraZero raises overall realism while jointly controlling vehicles, pedestrians, and cyclists in closed loop, where Gigaflow drives pedestrians with a scripted controller. The shaded row is the logged expert reference; **bold** marks the best demonstration-free entry per column and underline the second best.

Method	Realism Meta ↑	Lin. Speed ↑	Lin. Accel. ↑	Ang. Speed ↑	Ang. Accel. ↑	Dist. Obj. ↑	TTC ↑	Dist. Rd. Edge ↑	Off- road ↑	Coll. ↑
Expert Demonstration [31]	0.722	0.56	0.33	0.56	0.49	0.49	0.88	0.71	1.00	1.00
Random Agent [31]	0.155	0.00	0.04	0.07	0.12	0.00	0.73	0.18	0.29	0.00
Linear Extrapolation [31]	0.324	0.16	0.12	0.02	0.04	0.25	0.78	0.50	0.46	0.41
Stationary Policy [13]	0.501	0.04	0.06	0.50	0.32	0.06	0.74	0.24	0.86	0.95
Gigaflow (zero-shot) [13]	0.619	<u>0.26</u>	0.25	<u>0.51</u>	0.48	0.32	0.81	0.54	0.91	0.95
TerraZero (Ours, Waymo)	0.632	0.27	<u>0.23</u>	0.51	<u>0.46</u>	<u>0.27</u>	0.85	0.57	0.93	0.97
TerraZero (Ours, nuPlan, zero-shot)	<u>0.625</u>	0.22	<u>0.23</u>	<u>0.51</u>	<u>0.46</u>	0.26	<u>0.84</u>	<u>0.56</u>	<u>0.93</u>	<u>0.97</u>

heads over jerk (vehicles), unicycle (pedestrians), and compact-bicycle (cyclists) dynamics, trained on Waymo map geometry alone, without demonstration trajectories, on 32 A100 GPUs (Appendix E). A single checkpoint produces both the vehicle and the vulnerable-road-user rollouts reported below.

Baselines. We compare against representative imitation and self-play approaches, and we draw the 2024 peer numbers from the SPACeR paper [7], whose benchmark matches ours. The imitation family comprises SMART [49] and SMART fine-tuned with CAT-K [53], two tokenized models trained directly on WOMB demonstrations and reported as reference upper bounds. The self-play family comprises decentralized PPO trained on the task reward alone [27], Human-Regularized PPO (HR-PPO) [12], and SPACeR [7]. These methods differ in how much logged data they require: HR-PPO regularizes its policy toward a behavior-cloning reference and SPACeR anchors self-play to a pre-trained tokenized reference model through Kullback–Leibler (KL) alignment, so each depends on a separate reference policy trained on logged data even though neither consumes demonstrations in the self-play loss, whereas PPO and TerraZero use no logged human data of any kind. PPO, HR-PPO, and SPACeR also initialize agents and goals from logged trajectories, while TerraZero uses procedural random initialization. The 2023 edition adds the published Gigaflow results [13] alongside the random-agent, linear-extrapolation, and stationary lower bounds reported by the challenge organizers [31].

The 2024 edition scores vehicles and vulnerable road users (VRUs) separately, since their dynamics and behavioral patterns differ, so we report a vehicle table and a VRU table following the SPACeR protocol. WOSAC builds a per-feature negative-log-likelihood of the logged outcome under the simulated rollout distribution and aggregates the kinematic, interactive, and map-based likelihoods into a single realism composite, alongside the minimum average displacement error (minADE) of the rollouts. The Demo-Free column records how much logged data each method requires: a check mark for methods that use no logged human data at all, an open circle for methods that use no demonstrations directly but depend on a separate reference policy trained on logged data, and a cross for methods trained directly on demonstrations. The TerraZero entries come from a single heterogeneous checkpoint that controls vehicles, pedestrians, and cyclists jointly, while the SPACeR vehicle and VRU numbers come from two separately configured runs.

Discussion. Table 4 reports WOSAC 2023 against Gigaflow and the demonstration-free baselines. Unlike Gigaflow, whose WOSAC entry drives pedestrians with a scripted IDM-like controller and applies its learned policy only to vehicles and cyclists, TerraZero controls vehicles, pedestrians, and cyclists through one unified heterogeneous policy, which makes it suitable for unified, interactive

Table 5. WOSAC 2024 vehicle sim agent evaluation on the shared 880-scenario filtered validation subset. The realism, kinematic, interactive, and map columns are likelihood meta-scores (higher is better); minADE, collision, and off-road are rollout statistics (lower is better). The Demo-Free column uses ✓ for no logged human data, ◦ for an anchoring reference policy trained on logged data, and ✗ for direct demonstration training. Shaded rows are tokenized imitation-learning references; **bold** marks the best self-play entry per column and underline the second best.

Method	Demo Free	Realism ↑	Kinematic ↑	Interactive ↑	Map ↑	minADE ↓	Coll. ↓	Off-rd. ↓
SMART [49]	✗	0.720	0.450	0.725	0.870	1.84	0.170	0.130
CAT-K [53]	✗	0.766	0.490	0.792	0.890	1.47	0.060	0.090
PPO [27]	✓	0.710	0.327	0.751	<u>0.875</u>	12.73	0.038	0.053
HR-PPO [12]	◦	0.716	0.341	<u>0.756</u>	0.880	12.25	0.044	0.053
SPACeR [7]	◦	0.741	<u>0.411</u>	0.779	0.880	4.10	0.036	0.056
TerraZero (Ours, Waymo)	✓	<u>0.740</u>	0.412	0.702	0.786	<u>6.14</u>	0.007	0.036
TerraZero (Ours, nuPlan, zero-shot)	✓	0.732	0.392	0.698	0.778	7.99	<u>0.007</u>	<u>0.040</u>

Table 6. WOSAC 2024 VRU sim agent evaluation on the shared 880-scenario filtered validation subset. All columns are higher-is-better except minADE. The Demo-Free column uses ✓ for no logged human data, ◦ for an anchoring reference policy trained on logged data, and ✗ for direct demonstration training. **bold** marks the best self-play entry per column and underline the second best.

Method	Demo Free	Realism ↑	Kinematic ↑	Interactive ↑	Map ↑	minADE ↓
PPO [27]	✓	0.648	0.242	0.683	0.835	7.71
HR-PPO [12]	◦	0.668	0.285	<u>0.700</u>	<u>0.847</u>	7.01
SPACeR [7]	◦	0.729	0.413	0.762	0.866	2.07
TerraZero (Ours, Waymo)	✓	<u>0.683</u>	0.366	0.622	0.751	<u>2.88</u>
TerraZero (Ours, nuPlan, zero-shot)	✓	<u>0.683</u>	<u>0.369</u>	0.614	0.749	2.98

traffic-scene simulation. TerraZero edges Gigaflo on overall realism (0.632 versus 0.619) and leads on linear and angular speed, time-to-collision, road-edge distance, off-road, and collision, while trailing on distance-to-object and the acceleration kinematics.

Tables 5 and 6 report WOSAC 2024 against recent self-play methods. On vehicles, TerraZero matches SPACeR on the realism composite (0.740 versus 0.741) and the kinematic score (0.412 versus 0.411) and posts the lowest collision and off-road rates of any method, with weaker interactive and map likelihoods and a higher minADE. On VRUs, TerraZero reaches a realism composite of 0.683 that surpasses PPO (0.648) and HR-PPO (0.668) and trails SPACeR (0.729), with a VRU minADE (2.88) close to SPACeR (2.07) and far below PPO and HR-PPO. TerraZero reaches this without demonstrations, without an anchoring reference policy, and without logged-trajectory initialization, where SPACeR and HR-PPO each rely on a reference policy trained on logged data.

The same recipe transfers across datasets. A heterogeneous policy trained only on nuPlan map geometry, evaluated zero-shot on the Waymo WOSAC protocol with no Waymo training, nearly matches the Waymo-trained policy: vehicle realism 0.732 versus 0.740 and VRU realism 0.683 versus 0.683 under the 2024 protocol, and overall realism 0.625 versus 0.632 under the 2023 protocol (Tables 4, 5, and 6). This near-parity echoes the transfer study of Section 6.5 (Figure 6), where realism tracks the domain-randomization scheme rather than the training source; the WOSAC comparison here measures that same cross-dataset transfer under the full leaderboard protocol rather than the restricted transfer-matrix protocol.

The two benchmarks together support our central claim: TerraZero is not merely a fast simulator but a training substrate that yields policies competitive as both ego driving policies (Table 2) and sim agents (Tables 4, 5, and 6). A policy that closely mimics human kinematics may still fail on task metrics, while a policy can game val14 through overly cautious driving that diverges from human behavior; the two metrics therefore catch complementary failure modes.

6.5 Generalization

A central claim of our domain-randomization approach is that policy robustness derives from the randomization scheme rather than from memorizing dataset-specific patterns. We test this directly with a transfer study: a separate TerraZero policy is trained on each data source, each for *fewer training steps* than the main benchmark policies of Sections 6.3.1 and 6.4, and then evaluated, *zero-shot* with no fine-tuning on the target, on every other dataset and nuPlan city. We read each policy through two complementary lenses, distributional *realism* scored with the WOSAC vehicle protocol and ego *safety* scored with the NAVSIM v2 Extended PDM Score (EPDMS) [6]. Both lenses share the same five checkpoints and five targets, so together they yield a pair of transfer matrices whose rows are the training source and whose columns are the evaluation target (Figure 6).

Setup. The realism matrix scores each cell under the WOSAC 2024 protocol restricted to vehicles, reporting the vehicle realism meta-score [31]. The metric compares the distribution of simulated vehicle features against the logged ground truth over an 80-frame future horizon under 32 stochastic closed-loop rollouts, weighting kinematic, interactive, and map-based feature families. Agents are initialized from the logged state but pursue *randomly generated* goals, so the policy never observes the held-out logged destination. The safety matrix scores the same checkpoints with the NAVSIM v2 EPDMS [6, 14], reporting the core safety score, the minimum of the at-fault-filtered no-collision, drivable-area, driving-direction, and time-to-collision sub-metrics, so a single safety failure caps the cell. In this setup the policy controls only the ego vehicle while every other agent replays its logged trajectory. This isolates ego transfer behavior but leaves the surrounding traffic *non-reactive*, so the EPDMS score here measures ego safety against replayed agents rather than the fully reactive closed-loop driving of the nuPlan val14 benchmark in Table 2. Both matrices restore each checkpoint’s own training configuration and vary only the target maps, and each cell averages about 100 scenarios, which supports relative comparison rather than leaderboard-grade absolutes. Waymo realism cells use the official WOSAC subject set, while the nuPlan cities have no official subject set and so score all valid logged vehicles, giving WOSAC-2024-*style* rather than official-subject numbers.

Cross-dataset transfer. The first question is whether a policy pays a penalty for being evaluated away from the dataset it trained on. It does not, under either lens. Reading down any column of Figure 6, the score barely moves with the training source: the per-source mean realism spans only about 0.011 across the five policies (from 0.720 for the CARLA-trained policy to 0.731 for the Waymo-trained one), and the per-source mean safety spans about 0.018 (from 0.722 to 0.740), with no policy holding a consistent home-dataset advantage in either. A Waymo-trained policy scores 0.745 realism on Waymo, yet a nuPlan-trained policy reaches 0.713 there, and on nuPlan Singapore the Waymo-trained policy (0.753) edges the nuPlan-trained one (0.748). The CARLA-trained policy, whose training maps are just five synthetic towns, nonetheless lands in this same narrow band as the Waymo- and nuPlan-trained policies drawn from hundreds of thousands of scenes (0.720 realism and 0.722 safety, within 0.011 and 0.018 of the best), so transfer quality tracks the randomization scheme rather than the size of the training corpus. Both panels read as near-uniform color within each column: the dominant variation runs across columns, a property of the target, not across rows.

Cross-city transfer. Within nuPlan the same pattern holds at the level of individual cities. The policy trained only on Las Vegas and the policy trained on the US cities both transfer to held-out cities with

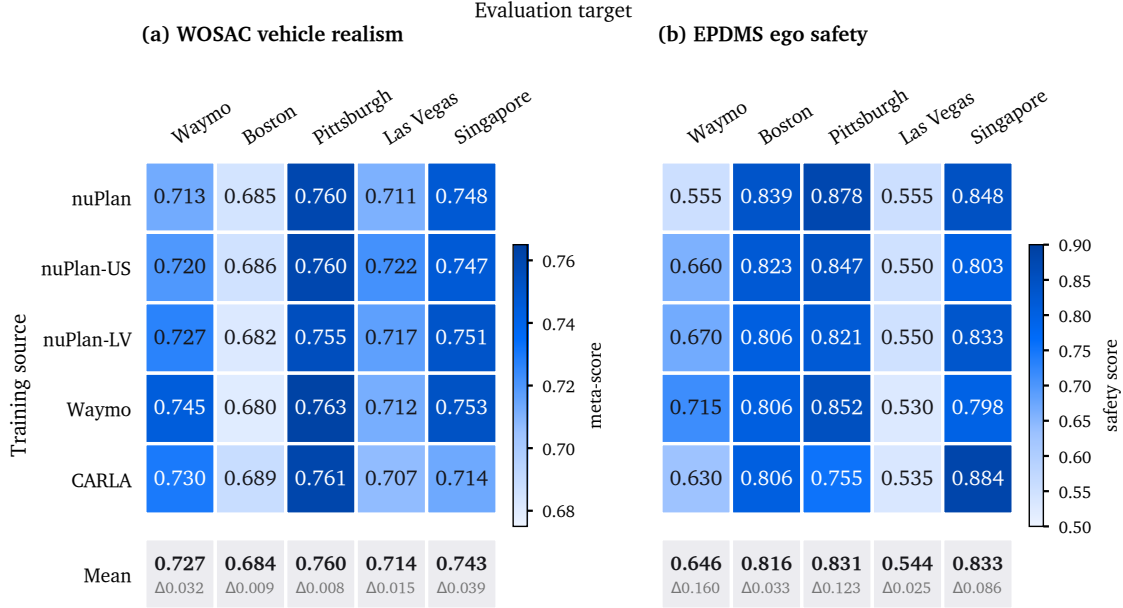


Fig. 6. Transfer matrices across datasets and cities. Each TerraZero policy trains on one source (row) and is scored zero-shot on each target (column), under (a) the WOSAC 2024 vehicle-realism meta-score and (b) the NAVSIM v2 EPDMS core ego-safety score, where darker is higher. Both panels band by column rather than by row, so a policy’s score is set by the evaluation target and not by the source it trained on. The bottom strip of each panel gives the per-target mean with the across-source spread (Δ , max minus min) below it. Row labels abbreviate the nuPlan US-cities and Las Vegas training splits; CARLA is a source only, since its maps carry no logged trajectories to score.

scores indistinguishable from the policy trained on the full nuPlan mix (Figure 6, bottom strips), even though Las Vegas supplies several times more training scenes than Boston or Singapore. Where the columns differ is between cities, not between sources: under realism, Boston sits lowest (0.684 mean) and Pittsburgh highest (0.760); under safety, Las Vegas is the uniformly low column (0.544 mean, spread only 0.025 across sources) while the nuPlan cities score high (0.82 to 0.83). The dominant training city thus earns no home advantage, and the safety score binds hardest on that very city. These are gaps of the target rather than of any policy: the two lenses disagree about which targets are hard, and the low realism on Boston and Pittsburgh is largely an artifact of road-edge geometry in the converted maps that every policy inherits equally, not a transfer penalty; on those two cities the off-road metric flags even logged human vehicles as off-road at rates several times the level seen elsewhere.

Emergent left-hand-traffic driving. The transfer matrices contain a built-in test of whether the policy memorizes a driving side or follows road structure. Four of the five training sources (Waymo, CARLA, the nuPlan US cities, and Las Vegas) contain only right-hand-traffic maps, yet nuPlan Singapore, a left-hand-traffic country, is among the highest-scoring targets under both metrics (a 0.743 realism column mean and a 0.833 safety column mean). The policy trained on the US cities, which never sees a left-hand-traffic map, scores 0.747 realism on Singapore, matching the 0.748 of the policy trained on the full nuPlan mix, whose training pool is the only source that includes Singapore itself. A policy that had memorized right-hand travel would collapse here; instead its realism and safety are undiminished, which indicates that the learned behavior is to follow the local lane topology rather than a fixed directional convention. We attribute this to kinematic domain randomization combined with reward-conditioned training, which reward progress along the road structure the agent is

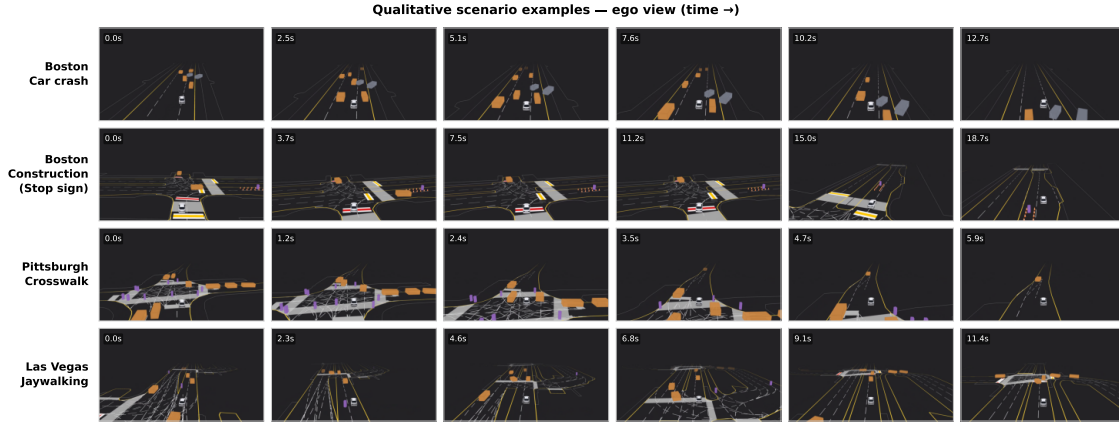


Fig. 7. Qualitative ego-policy rollouts. Four safety-critical scenario types and cities (top to bottom: a Boston car crash, a Boston stop-sign construction zone, a Pittsburgh crosswalk, and a Las Vegas jaywalking pedestrian). Each row is a single episode shown as a left-to-right time sequence in the ego view; the TerraZero policy sustains progress while avoiding collisions and yielding to vulnerable road users.

placed on. As noted by Wang et al. [48], achieving such transfer without human demonstrations is a key indicator of genuine robustness.

6.6 Qualitative Analysis

Beyond aggregate metrics, we visualize policy behavior in the situations that most stress a driving policy. Figure 7 shows ego-view rollouts of a single TerraZero policy across four safety-critical scenario types drawn from different cities: a car-crash encounter in Boston, a stop-sign construction zone in Boston, a pedestrian crosswalk in Pittsburgh, and a jaywalking pedestrian in Las Vegas. Each row reads left to right as the episode advances, with the policy-controlled ego agent navigating the hazard while reactive traffic and pedestrians evolve around it. The policy keeps progress through congestion, respects the stop-sign and crosswalk geometry, and yields to the jaywalking pedestrian rather than colliding. These behaviors are consistent with the quantitative gains in the No AF-Collision and TTC components of Table 2. Video rollouts of these and additional scenarios are available on the [project website](#), covering procedurally generated long-tail scenarios for cars and trucks, heterogeneous multi-agent simulation, closed-loop driving on val14 and InterPlan, and traffic simulation on WOSAC.

7 Conclusion

We have presented TerraZero, a high-performance closed-loop driving simulator designed for self-play reinforcement learning at scale. TerraZero addresses a persistent tension in autonomous driving research: the simulators fast enough for RL-scale training have historically lacked the scenario fidelity needed for meaningful policy transfer, while feature-rich environments remain orders of magnitude too slow. By combining a configurable C simulation engine, a procedural scenario generator, and a compute-efficient self-play recipe, TerraZero achieves among the highest reported throughput for object-level, self-play-enabled driving simulators while supporting heterogeneous traffic (vehicles, pedestrians, cyclists), multiple dynamics models, traffic-rule enforcement, and multi-source map data from Waymo, nuPlan, and CARLA.

The system makes three concrete contributions: a fast, feature-rich object-level C simulator that retains full scenario fidelity at high throughput across consumer-grade, server-grade, and multi-GPU

hardware; a procedural scenario generator that manufactures a vast space of scenarios from real-world maps rather than training on the logs directly; and a compute-efficient self-play recipe that trains every reported policy from scratch with zero human demonstrations, using no imitation and no logged trajectories, and generalizes zero-shot across cities and datasets. We measure driving performance on nuPlan val14 and InterPlan, and sim-agent realism on WOSAC. The same recipe tops the long-tail InterPlan benchmark, ranks among the best-performing approaches on the routine-driving val14 benchmark and is the safest there by collision and time-to-collision, and outperforms other demonstration-free methods on WOSAC realism, so one stack yields both a realistic traffic simulator and a high-performance, robust planner.

Limitations. TerraZero relies on high-definition maps with lane-level topology, traffic signal phase information, and intersection geometry; regions for which such maps are unavailable cannot be used as training or evaluation scenarios. The object-level simulation abstraction does not model visual perception: policies operate on ground-truth structured features rather than camera or lidar inputs, so the system cannot directly train end-to-end perception-to-control pipelines. While domain randomization over kinematic parameters improves robustness, the sim-to-real gap for physical vehicle dynamics (tire friction, suspension response, aerodynamic effects) remains an open challenge that randomization alone does not fully resolve.

Future Work. Several directions extend naturally from the current system. *City-scale simulation* would increase scenario duration and spatial extent to support long-horizon planning across entire urban networks, moving beyond the intersection-level episodes that currently dominate training. *Expanded agent types* (emergency vehicles, construction equipment, electric scooters, and other underrepresented road users) would enrich interaction dynamics and further stress-test policy generalization. *Sim-to-real transfer experiments* leveraging the kinematic domain randomization framework would provide direct evidence of whether policies trained in TerraZero transfer to physical platforms, closing the loop between simulation and deployment. Finally, *integration with visual perception pipelines* (replacing ground-truth observations with learned representations from camera or lidar inputs) would bridge the gap between object-level and end-to-end autonomous driving, enabling TerraZero to serve as a training environment for full-stack driving systems.

References

- [1] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalam-barkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhres, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 929–947. doi:10.1145/3620665.3640366
- [2] Trapit Bansal, Jakub Pachocki, Szymon Sidor, Ilya Sutskever, and Igor Mordatch. 2018. Emergent Complexity via Multi-Agent Competition. In *International Conference on Learning Representations (ICLR)*.
- [3] Luca Bergamini, Yawei Ye, Oliver Scheel, Long Chen, Chih Hu, Luca Del Pero, Blazej Osinski, Hugo Grimmett, and Peter Ondruska. 2021. SimNet: Learning Reactive Self-Driving Simulations from Real-World Observations. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*. 5119–5125.
- [4] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, et al. 2019. Dota 2 with Large Scale Deep Reinforcement Learning. *arXiv preprint arXiv:1912.06680* (2019).
- [5] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. 2021. nuPlan: A Closed-Loop ML-Based Planning Benchmark for Autonomous Vehicles. *arXiv preprint arXiv:2106.11810* (2021).

- [6] Wei Cao, Marcel Hallgarten, Tianyu Li, Daniel Dauner, Xunjiang Gu, Caojun Wang, Yakov Miron, Marco Aiello, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. 2025. Pseudo-Simulation for Autonomous Driving. In *Conference on Robot Learning (CoRL)*.
- [7] Wei-Jer Chang, Akshay Ranges, Kevin Joseph, Matthew Strong, Masayoshi Tomizuka, Yihan Hu, and Wei Zhan. 2026. SPACeR: Self-Play Anchoring with Centralized Reference Models. In *International Conference on Learning Representations (ICLR)*.
- [8] Jie Cheng, Yingbing Chen, and Qifeng Chen. 2024. PLUTO: Pushing the Limit of Imitation Learning-based Planning for Autonomous Driving. *arXiv preprint arXiv:2404.14327* (2024).
- [9] Jie Cheng, Yingbing Chen, Xiaodong Mei, Bowen Yang, Bo Li, and Ming Liu. 2024. Rethinking Imitation-based Planners for Autonomous Driving (PlanTF). In *International Conference on Robotics and Automation (ICRA)*.
- [10] Daphne Cornelisse, Spencer Cheng, Pragnay Mandavilli, Julian Hunt, Kevin Joseph, Wael Doulazmi, Valentin Charraut, Aditya Gupta, Joseph Suárez, and Eugene Vinitsky. 2025. PufferDrive: A Fast and Friendly Driving Simulator for Training and Evaluating RL Agents. GitHub repository. <https://github.com/Emerge-Lab/PufferDrive>
- [11] Daphne Cornelisse, Aarav Pandya, Kevin Joseph, Joseph Suárez, and Eugene Vinitsky. 2025. Building Reliable Sim Driving Agents by Scaling Self-Play. *arXiv preprint arXiv:2502.14706* (2025).
- [12] Daphne Cornelisse and Eugene Vinitsky. 2024. Human-Compatible Driving Agents Through Data-Regularized Self-Play Reinforcement Learning. *Reinforcement Learning Journal* 5 (2024), 2320–2344.
- [13] Marco Cusumano-Towner, David Hafner, Alexander Hertzberg, Brody Huval, Aleksei Petrenko, Eugene Vinitsky, Erik Wijmans, Taylor W. Killian, Stuart Bowers, Ozan Sener, Philipp Kraehenbuehl, and Vladlen Koltun. 2025. Robust Autonomy Emerges from Self-Play. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 11710–11737.
- [14] Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. 2023. Parting with Misconceptions about Learning-based Vehicle Motion Planning. In *Conference on Robot Learning (CoRL)*.
- [15] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinsuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. 2024. NAVSIM: Data-Driven Non-Reactive Autonomous Vehicle Simulation and Benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [16] Aron Distelzweig, Faris Janjoš, Andreas Look, Anna Rothenhäusler, Daniel Jost, Oliver Scheel, Raghu Rajan, Daphne Cornelisse, Eugene Vinitsky, and Joschka Boedecker. 2026. Beyond Self-Play and Scale: A Behavior Benchmark for Generalization in Autonomous Driving. *arXiv preprint arXiv:2605.10034* (2026).
- [17] Aron Distelzweig, Faris Janjoš, Oliver Scheel, Sirish Reddy Varra, Raghu Rajan, and Joschka Boedecker. 2026. Perfect Prediction or Plenty of Proposals? What Matters Most in Planning for Autonomous Driving. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2510.15505.
- [18] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. 2017. CARLA: An Open Urban Driving Simulator. In *Proceedings of the 1st Annual Conference on Robot Learning (CoRL) (Proceedings of Machine Learning Research, Vol. 78)*. PMLR, 1–16.
- [19] Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. 2018. IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, 1407–1416.
- [20] Scott Ettinger, Shuyang Cheng, Benjamin Caine, Chenxi Liu, Hang Zhao, Sabeek Pradhan, Yuning Chai, Ben Sapp, Charles R. Qi, Yin Zhou, Zoey Yang, Aurélien Chouard, Pei Sun, Jiquan Ngiam, Vijay Vasudevan, Alexander McCauley, Jonathon Shlens, and Dragomir Anguelov. 2021. Large Scale Interactive Motion Forecasting for Autonomous Driving: The Waymo Open Motion Dataset. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 9710–9719.
- [21] Cole Gulino, Justin Fu, Wenjie Luo, George Tucker, Eli Bronstein, Yiren Lu, Jean Harb, Xinlei Pan, Yan Wang, Xiangyu Chen, John D. Co-Reyes, Rishabh Agarwal, Rebecca Roelofs, Yao Lu, Nico Montali, Paul Mougin, Zoey Yang, Brandyn White, Aleksandra Faust, Rowan McAllister, Dragomir Anguelov, and Benjamin Sapp. 2023. Waymax: An Accelerated, Data-Driven Simulator for Large-Scale Autonomous Driving Research. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- [22] Marcel Hallgarten, Martin Stoll, and Andreas Zell. 2023. From Prediction to Planning with Goal Conditioned Lane Graph Traversals. In *IEEE International Conference on Intelligent Transportation Systems (ITSC)*.
- [23] Marcel Hallgarten, Julian Zapata, Martin Stoll, Katrin Renz, and Andreas Zell. 2024. Can Vehicle Motion Planning Generalize to Realistic Long-tail Scenarios?. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- [24] Zhiyu Huang, Peter Karkus, Boris Ivanovic, Yuxiao Chen, Marco Pavone, and Chen Lv. 2024. DTPP: Differentiable Joint Conditional Prediction and Cost Evaluation for Tree Policy Planning in Autonomous Driving. In *IEEE International Conference on Robotics and Automation (ICRA)*.

- [25] Zhiyu Huang, Haochen Liu, and Chen Lv. 2023. GameFormer: Game-theoretic Modeling and Learning of Transformer-based Interactive Prediction and Planning for Autonomous Driving. In *IEEE/CVF International Conference on Computer Vision (ICCV)*.
- [26] Bernhard Jaeger, Daniel Dauner, Jens Beißwenger, Simon Gerstenecker, Kashyap Chitta, and Andreas Geiger. 2025. CaRL: Learning Scalable Planning Policies with Simple Rewards. In *Conference on Robot Learning (CoRL)*.
- [27] Saman Kazemkhani, Aarav Pandya, Daphne Cornelisse, Brennan Shacklett, and Eugene Vinitzky. 2025. GPU-Drive: Data-Driven, Multi-Agent Driving Simulation at 1 Million FPS. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [28] Alex Kendall, Jeffrey Hawke, David Janz, Przemyslaw Mazur, Daniele Reda, John-Mark Allen, Vinh-Dieu Lam, Alex Bewley, and Amar Shah. 2019. Learning to Drive in a Day. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 8248–8254.
- [29] Arne Kesting, Martin Treiber, and Dirk Helbing. 2007. General Lane-Changing Model MOBIL for Car-Following Models. *Transportation Research Record* 1999, 1 (2007), 86–94.
- [30] Quanyi Li, Zhenghao Peng, Lan Feng, Zhizheng Liu, Chenda Duan, Wenjie Mo, and Bolei Zhou. 2023. ScenarioNet: Open-Source Platform for Large-Scale Traffic Scenario Simulation and Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [31] Nico Montali, John Lambert, Paul Mougin, Alex Kuefler, Nick Rhinehart, Michelle Li, Cole Gulino, Tristan Emrich, Zoey Yang, Shimon Whiteson, Brandyn Allen White, and Dragomir Anguelov. 2023. The Waymo Open Sim Agents Challenge. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 7730–7742.
- [32] National Electrical Manufacturers Association. 2003. NEMA Standards Publication TS 2-2003 (R2008): Traffic Controller Assemblies with NTCIP Requirements, Version 02.06. NEMA Standards Publication.
- [33] Fabio Pardo, Arash Tavakoli, Vitaly Levdiv, and Petar Kormushev. 2018. Time Limits in Reinforcement Learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, 4045–4054.
- [34] Jonah Philion, Xue Bin Peng, and Sanja Fidler. 2024. Trajenglish: Traffic Modeling as Next-Token Prediction. In *The Twelfth International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=Z59Rb5bPPP>
- [35] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. 2016. Prioritized Experience Replay. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [36] Oliver Scheel, Luca Bergamini, Maciej Wolczyk, Błażej Osiński, and Peter Ondruska. 2022. Urban Driver: Learning to Drive from Real-world Demonstrations Using Policy Gradients. In *Conference on Robot Learning (CoRL)*. 718–728.
- [37] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2016. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [38] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [39] Adam Scibior, Vasileios Lioutas, Daniele Reda, Peyman Bateni, and Frank D. Wood. 2021. Imagining The Road Ahead: Multi-Agent Trajectory Prediction via Differentiable Simulation. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*. 720–725.
- [40] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. 2017. Mastering the Game of Go without Human Knowledge. *Nature* 550, 7676 (2017), 354–359.
- [41] Joseph Suarez. 2024. PufferLib: Making Reinforcement Learning Libraries and Environments Play Nice. *arXiv preprint arXiv:2406.12905* (2024).
- [42] Tianyi Tan, Yinan Zheng, Ruiming Liang, Zexu Wang, Kexin Zheng, Jinliang Zheng, Jianxiong Li, Xianyu Zhan, and Jingjing Liu. 2025. Flow Matching-Based Autonomous Driving Planning with Advanced Interactive Behavior Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2510.11083.
- [43] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. 2017. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 23–30.
- [44] Martin Treiber, Ansgar Hennecke, and Dirk Helbing. 2000. Congested Traffic States in Empirical Observations and Microscopic Simulations. *Physical Review E* 62 (2000), 1805–1824.
- [45] Hado van Hasselt, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. 2016. Learning Values Across Many Orders of Magnitude. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 29. 4287–4295.
- [46] Eugene Vinitzky, Nathan Lichtlé, Xiaomeng Yang, Brandon Amos, and Jakob Foerster. 2022. Nocturne: A Scalable Driving Benchmark for Bringing Multi-Agent Learning One Step Closer to the Real World. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- [47] Lingguang Wang, Ömer Şahin Taş, Marlon Steiner, and Christoph Stiller. 2025. FlowDrive: Moderated Flow Matching with Data Balancing for Trajectory Planning. *arXiv preprint arXiv:2509.21961* (2025).
- [48] Zilin Wang, Saeed Rahmani, Daphne Cornelisse, Bidipta Sarkar, Alexander David Goldie, Jakob Nicolaus Foerster, and Shimon Whiteson. 2026. Learning to Drive in New Cities Without Human Demonstrations. *arXiv preprint arXiv:2602.15891* (2026).
- [49] Wei Wu, Xiaoxin Feng, Ziyang Gao, and Yuheng Kan. 2024. SMART: Scalable Multi-agent Real-time Motion Generation via Next-token Prediction. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [50] Zhouchonghao Wu, Raymond Song, Vedant Mundheda, Luis E. Navarro-Serment, Christof Schoenborn, and Jeff Schneider. 2026. TADPO: Reinforcement Learning Goes Off-road. *arXiv preprint arXiv:2603.05995* (2026).
- [51] Danfei Xu, Yuxiao Chen, Boris Ivanovic, and Marco Pavone. 2023. BITS: Bi-Level Imitation for Traffic Simulation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 2929–2936.
- [52] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan R Salakhutdinov, and Alexander J Smola. 2017. Deep Sets. In *Advances in Neural Information Processing Systems (NeurIPS)*. 3391–3401.
- [53] Zhejun Zhang, Peter Karkus, Maximilian Igl, Wenhao Ding, Yuxiao Chen, Boris Ivanovic, and Marco Pavone. 2025. Closed-Loop Supervised Fine-Tuning of Tokenized Traffic Models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [54] Zhejun Zhang, Alexander Liniger, Dengxin Dai, Fisher Yu, and Luc Van Gool. 2023. TrafficBots: Towards World Models for Autonomous Driving Simulation and Motion Prediction. In *International Conference on Robotics and Automation (ICRA)*.
- [55] Yinan Zheng, Ruiming Liang, Kexin Zheng, Jinliang Zheng, Liyuan Mao, Jianxiong Li, Weihao Gu, Rui Ai, Shengbo Eben Li, Xianyu Zhan, and Jingjing Liu. 2025. Diffusion-Based Planning for Autonomous Driving with Flexible Guidance. In *International Conference on Learning Representations (ICLR)*. arXiv:2501.15564.
- [56] Ziyuan Zhong, Davis Rempe, Yuxiao Chen, Boris Ivanovic, Yulong Cao, Danfei Xu, Marco Pavone, and Baishakhi Ray. 2023. Language-Guided Traffic Simulation via Scene-Level Diffusion. In *Conference on Robot Learning (CoRL)*.
- [57] Ming Zhou, Jun Luo, Julian Villeda, Yaodong Yang, David Rusu, Jiayu Miao, Weinan Zhang, Montgomery Alban, Iman Fadar, Zheng Chen, et al. 2021. SMARTS: An Open-Source Scalable Multi-Agent RL Training School for Autonomous Driving. In *Proceedings of the 2020 Conference on Robot Learning (Proceedings of Machine Learning Research, Vol. 155)*. PMLR, 264–285.

A Simulation Constants

Table 7 lists the structural parameters that define the simulation environment’s capacity and behavior.

Table 7. Simulation constants. Compile-time limits govern memory allocation; runtime parameters are configurable per experiment.

Category	Parameter	Value
<i>Observations</i>		
	Max observed partner agents	20
	Max observed road segments	200
	Max observed traffic entities	16
	Default partner observation radius	50 m
	Default traffic observation radius	100 m
<i>Time</i>		
	Simulation timestep (Δt)	0.1 s
	Policy timestep	0.1 s
	Episode length	256 steps (25.6 s)
<i>Map Structure</i>		
	Max phases per intersection	8
	Max signals per intersection	16
	Max stop lines per intersection	16
	Max travel directions per lane	32
	Z-separation threshold (multi-level filtering)	2.5 m

B Reward Function Details

Throughout this appendix, a *Used by* tag records whether a setting applies to both reported policies of Section 6.2 (Both), only the planner (Planner), or only the heterogeneous sim agent (Sim agent).

Variable definitions.

- p, p_g : agent position and goal position in world frame.
- δ_{goal} : goal region radius (default 2.0 m; randomizable in $[2, 12]$ m).
- v_{goal} : maximum speed for goal acceptance (default 3.0 m/s). An agent must be within δ_{goal} and traveling below v_{goal} to register goal achievement, preventing drive-through exploits.
- γ_{col} : collision speed scale (default 0.1), controlling how strongly speed amplifies the collision and red-light penalties. At $v = 20$ m/s the penalty roughly doubles compared to a stationary collision.
- n_{viol} : count of comfort threshold violations per timestep. Four thresholds are checked independently: $|a_{\text{long}}| > 3.0 \text{ m/s}^2$, $|a_{\text{lat}}| > 3.0 \text{ m/s}^2$, $|j_{\text{long}}| > 5.0 \text{ m/s}^3$, $|j_{\text{lat}}| > 5.0 \text{ m/s}^3$. The penalty is proportional to the number of violations ($n_{\text{viol}} \in \{0, 1, 2, 3, 4\}$), not their magnitude. The thresholds are per agent type; the values above are for vehicles, while pedestrians use (1.5, 1.0, 12.0, 5.0) and cyclists (3.0, 1.5, 40.0, 18.0) for $(|a_{\text{long}}|, |a_{\text{lat}}|, |j_{\text{long}}|, |j_{\text{lat}}|)$.
- θ_f : heading deviation from the nearest lane’s travel direction (in radians).
- x_f : signed lateral offset from lane center (in meters). b_{center} is a configurable center bias (default 0.0) that shifts the preferred lateral position, e.g., for right-of-lane preference.
- v_{max} : maximum velocity after kinematic scaling, $v_{\text{max}} = 20.0 \cdot c_{\text{vel}}$ m/s. The base clip is 20.0 m/s; the per-agent coefficient $c_{\text{vel}} \in [0.5, 1.5]$ modulates it.
- v_{lim} : speed-limit cap for the sim agent’s speed-limit term, resolved per lane from the posted limit when one is available.

Table 8. Reward terms and their per-type weights. Each term is computed per timestep per agent; entries are the per-agent randomization range [min, max] or a fixed value, and a dash marks a term disabled for that type. The planner controls vehicles only and so uses the Vehicle column, while the heterogeneous sim agent uses all three. Goal progress and target speed are held at zero across both reported policies and are omitted, so the full parameter vector has 23 entries (Appendix C). The auxiliary thresholds δ_{goal} , v_{goal} , b_{center} , and $v_{\text{speed_lim}}$ are defined below.

Term	Formula	Category	Vehicle	Pedestrian	Cyclist
Goal	$+\alpha_{\text{goal}}$ if $\ p - p_g\ < \delta_{\text{goal}}$ and $v < v_{\text{goal}}$	Navigation	1.0	1.0	1.0
Collision	$-(\alpha_{\text{col}} + \gamma_{\text{col}} \cdot v)$	Safety	[0.0, 3.0]	[0.5, 2.0]	[1.0, 3.0]
Boundary	$-\alpha_{\text{bound}}$ if off drivable area	Safety	[0.0, 3.0]	—	[0.5, 2.0]
Comfort	$-\alpha_{\text{comf}} \cdot n_{\text{viol}}$	Comfort	[0.0, 0.1]	—	—
Lane Align	$-\alpha_{\text{align}} \cdot \theta_f $	Lane	[2.5e-4, 2.5e-2]	—	[5e-3, 2e-2]
Lane Center	$-\alpha_{\text{center}} \cdot x_f + b_{\text{center}} $	Lane	[2.5e-4, 7.5e-3]	—	—
Velocity	$+\alpha_{\text{vel}} \cdot \Delta t \cdot v / v_{\text{max}}$	Speed	2.5e-3	—	—
Vel Align	$-\alpha_{\text{va}} \cdot \Delta v $	Speed	[0.0, 1.0]	—	[0.25, 0.75]
Reverse	$-\alpha_{\text{rev}}$ if $v < 0$	Speed	[2.5e-4, 7.5e-3]	—	—
Timestep	$+\alpha_{\text{step}}$ (survival)	Speed	2.5e-5	—	—
Stop Line	$-\alpha_{\text{stop}}$ if violating	Compliance	5.0	—	5.0
Red Light	$-(\alpha_{\text{red}} + \gamma_{\text{col}} \cdot v)$	Compliance	3.0	—	3.0
Road Incursion	$-\alpha_{\text{incur}} \cdot \Delta t$ if off-road and off-crosswalk	Safety	—	[0.5, 2.0]	—
Speed Limit	$-\alpha_{\text{slim}} \cdot \Delta t \cdot (v - v_{\text{lim}}) / v_{\text{max}}$ if $v > v_{\text{lim}}$	Speed	—	[0.5, 2.0]	[0.25, 1.0]
Edge Preference	$+\alpha_{\text{edge}} \cdot \Delta t \cdot$ $\text{clamp}(x_f / w_{\text{lane}})$	Lane	—	—	[0.01, 0.05]

- $v_{\text{speed_lim}}$: per-type speed-limit parameter sampled per agent (pedestrians [1.5, 2.5], cyclists [5.0, 8.0]; disabled for vehicles).
- Δt : simulation timestep (0.1 s). The velocity and speed-based rewards are multiplied by Δt to produce per-step rather than per-second rewards.

Reward parameter randomization. Each of the 23 reward parameters supports three modes: a **fixed scalar** (constant across all agents and episodes), a [min, max] **pair** (uniformly sampled per agent at episode start), or **null** (excluded from reward computation, and from the observation vector when null for every agent type). The per-type weights are those of Table 8: the planner uses the Vehicle column, while the heterogeneous sim agent uses all three.

Sampled parameters are normalized to $[-1, 1]$ via $\hat{\alpha}_i = 2(\alpha_i - \alpha_i^{\min}) / (\alpha_i^{\max} - \alpha_i^{\min}) - 1$ before inclusion in the ego observation vector. Fixed parameters (where $\alpha_i^{\min} = \alpha_i^{\max}$) normalize to 0.

C Observation Space Details

The observation space is organized into four groups, each encoded via Deep Sets [52] (a shared MLP per entity, followed by max-pooling to produce a fixed-length vector regardless of entity count). All observations are computed in the **ego agent’s local coordinate frame** and zero-padded when fewer than the maximum number of entities are present. Each feature can be independently enabled or disabled, and the network input resizes to the enabled set.

Ego-centric coordinate transform. All spatial features (positions, headings, velocities) are expressed relative to the ego agent. Given a world-frame displacement $(\Delta x, \Delta y)$ and the ego agent’s heading unit vector $(\cos \psi, \sin \psi)$, the ego-frame coordinates are:

$$x_{\text{ego}} = \Delta x \cos \psi + \Delta y \sin \psi, \quad y_{\text{ego}} = -\Delta x \sin \psi + \Delta y \cos \psi, \quad (1)$$

where x_{ego} points forward along the ego heading and y_{ego} points left. This transform is applied to goal positions, partner positions, road segment midpoints, and traffic entity positions.

The active reward weight parameters are included in the ego observation vector (**reward-conditioned observations**), enabling a single policy to generalize across reward configurations: adjusting the weight vector at inference time changes driving behavior without retraining. The planner exposes 19 of the 23 parameters (its four vulnerable-road-user terms are null), while the heterogeneous sim agent exposes all 23, since no parameter is null across its three agent types.

Table 9. Ego features for the planner (1 agent). Features marked with † are optional and controlled by configuration flags. The heterogeneous sim agent exposes all 23 reward parameters in its `alpha_params` block (none are null across its three types).

Feature	Dims	Computation
<code>agent_type</code>	1	Categorical index: vehicle=1, pedestrian=2, cyclist=3
<code>rel_goal_pos</code>	2	Ego-frame goal position (Eq. 1), scaled $\times 0.005$
<code>goal_dropout_flag</code>	1	Binary: 1 when the goal is hidden this step (goal dropout)
<code>state_dropout_flag</code>	1	Binary: 1 when ego state features are dropped (robustness)
<code>speed</code>	1	$\text{sign}(\mathbf{v} \cdot \hat{\mathbf{h}}) \sqrt{v_x^2 + v_y^2} / 100$ (signed; negative when reversing)
<code>vehicle_dims</code>	2	(width / 15, length / 30)
<code>collision_state</code>	1	Binary: 1 if currently in collision
<code>steering_angle</code>	1	$\delta_{\text{steer}} / \pi$
<code>acceleration</code>	2	$(a_{\text{long}}/5, a_{\text{lat}}/4)$
<code>stop_sign_state</code> †	3	One-hot: (not in region, must stop, cleared); on with traffic rules
<code>alpha_params</code> †	19	Active reward params normalized to $[-1, 1]$ (Sec. B)
<code>kinematic_params</code> †	4	$(c_{\text{throttle}}, c_{\text{steer}}, c_{\text{acc}}, c_{\text{vel}})$ normalized

The engine also implements ego features that are disabled in the default configuration: lane state $(\theta_f/\pi, x_f/2.5)$, topology distance to goal, local road curvature, lateral velocity, and yaw rate.

Table 10. Partner features (nearest agents within the observation radius).

Feature	Dims	Computation
<code>rel_pos</code>	2	Ego-frame position (Eq. 1), scaled $\times 0.02$
<code>vehicle_dims</code>	2	(width / 15, length / 30)
<code>rel_heading</code>	2	$(\cos \Delta\psi, \sin \Delta\psi)$ relative to ego heading
<code>speed</code>	1	$\sqrt{v_x^2 + v_y^2} / 100$
<code>actor_type</code>	1	Categorical: vehicle / pedestrian / cyclist
<code>acceleration</code> †	2	$(a_{\text{long}}/5, a_{\text{lat}}/4)$; disabled by default
Total per partner	8	

Partners are the closest agents within the observation radius, filtered by elevation so that agents on a different road level are ignored.

Road segments fill a fixed budget that prioritizes lane centerlines and road edges, the geometry most relevant to routing and to the off-road boundary, over lane-boundary markings, so dense markings

Table 11. Road features (nearest road segments).

Feature	Dims	Computation
rel_pos	2	Ego-frame midpoint of segment, scaled $\times 0.02$
segment_dims	2	(length / 100, width / 100)
rel_angle	2	$(\cos \Delta\phi, \sin \Delta\phi)$: segment direction in ego frame
road_type	1	Categorical: lane=0, line=1, edge=2
Total per segment	7	

cannot crowd the essential geometry out of the budget. Segments are filtered by elevation to handle multi-level road structures (bridges, overpasses).

Table 12. Traffic features (nearby traffic entities).

Feature	Dims	Computation
entity_type	1	0 = stop line, 1 = traffic light
rel_pos	3	Ego-frame 3D position, scaled (0.02, 0.02, 0.1)
signal_state	4	One-hot: (red, yellow, green, off/unknown)
stop_line_endpoints	4	Ego-frame stop-line segment endpoints, scaled $\times 0.02$
Total per entity	12	

Traffic entities are the stop lines nearest the ego, selected by ascending distance up to the budget, so the ego’s own approaching stop line, its sole channel to the signal state, is never crowded out on dense maps.

D Agent Initialization

At each episode reset, the simulation initializes agents through a multi-step procedure that composes scenario data with several randomization mechanisms to produce diverse, challenging traffic configurations. Algorithm 1 summarizes the full initialization sequence.

Algorithm 1 Agent initialization at episode reset.

- 1: Load terrabin scenario (map geometry, logged trajectories, signal phases)
- 2: Compute world-frame mean $(\bar{x}, \bar{y})$ and center all coordinates
- 3: Initialize grid map and lane connectivity graph
- 4: **if** random initialization **then**
- 5: **Random placement:** sample agent count, place on lanes (Sec. D.1)
- 6: **else**
- 7: **Log initialization:** load positions from recorded trajectories
- 8: **end if**
- 9: **Role assignment:** assign policy-controlled, log-replay, or IDM roles (Sec. D.2)
- 10: **for** each active agent **do**
- 11: Sample kinematic parameters $(c_{\text{throttle}}, c_{\text{steer}}, c_{\text{acc}}, c_{\text{vel}})$ (Sec. D.4)
- 12: Sample reward parameters $(\alpha_1, \dots, \alpha_{23})$ (Sec. B, Table 8)
- 13: Initialize velocity and heading from trajectory data or defaults
- 14: Reset collision state, metrics, and displacement tracking
- 15: **end for**
- 16: **Goal assignment:** set navigation goals from scenario data or topology sampling (Sec. D.5)

D.1 Random Agent Placement

When random initialization is enabled, the agent count is sampled from a configurable range whose upper bound may exceed the number of agents in the source scenario’s logged data, which lets scenes pack denser than any naturalistic recording. Agents are then placed onto the lane network under rejection sampling: each placement starts on a lane, aligned with its travel direction and at low speed, and is retried until it is collision-free and on the road. Lanes inside signalized intersections are excluded so agents do not begin mid-intersection. Bounding-box dimensions are sampled independently per agent from type-specific uniform ranges.

D.2 Behavioral Role Assignment

After placement, each agent is assigned a behavioral role based on the configured control mode, which selects the agent types under policy control: from the ego vehicle alone, through vehicles only (the planner setting), up to every vehicle, pedestrian, and cyclist (the sim-agent setting). Candidates are filtered so that only agents with a valid lane and a sufficiently distant goal enter policy control, ensuring a non-trivial navigation task; the remaining agents follow recorded trajectories or rule-based controllers as background traffic.

D.3 Hybrid Initialization

Hybrid initialization provides a middle ground between fully random and fully log-based initialization. A configurable probability $p_{\text{random}} \in [0, 1]$ controls the per-environment decision: with probability p_{random} , agents in that environment are randomly placed; otherwise, they are initialized from logged trajectories at timestep 0. When initialized from logs, agents begin at their recorded position, heading, and velocity but immediately transition to policy control: the log provides only the initial condition, not the ongoing behavior.

This mode is particularly useful for bootstrapping training: logged initial conditions place agents in coherent, mid-traffic states (e.g., at highway speed in a merge lane) that random placement would rarely produce, accelerating the policy’s exposure to interactive situations, at the cost of the state diversity that random placement provides.

D.4 Kinematic Parameter Randomization

At episode start, four multiplicative coefficients are sampled per agent and held constant for the episode’s duration. Each coefficient scales the corresponding dynamics parameter:

Table 13. Kinematic randomization coefficients, each randomized per agent between 0.5 and 1.5.

Coefficient	Scales	Effect across the range
c_{throttle}	Longitudinal jerk input	More/less responsive acceleration
c_{steer}	Lateral jerk/steering input	Tighter/looser turning
c_{acc}	Max longitudinal acceleration clip	Higher/lower accel. limits
c_{vel}	Max velocity clip ($v_{\text{max}} = 20 \cdot c_{\text{vel}}$ m/s)	Faster/slower max speed

Velocity and acceleration bounds, scaled by the respective coefficients, are applied after the dynamics integration step, ensuring physical plausibility regardless of the sampled coefficients.

All four coefficients are included in the ego observation vector (Table 9), normalized to $[-1, 1]$ using this range, so the policy can condition its behavior on the current dynamics regime online.

D.5 Goal Assignment

Each agent requires a navigation goal for the goal-achievement reward and the goal-relative observation. Goals are assigned differently depending on the initialization mode:

Log-based goals. When using logged trajectories, the goal is set to the agent’s final recorded position in the source scenario, verified to be reachable via the lane connectivity graph before the scenario enters the training pool.

Random goal generation. When random initialization is used, or when an agent that reached its goal receives a new one, goals are sampled by a forward walk along the lane connectivity graph, following random candidate lanes at junctions, up to a target arc length drawn from a bounded range. An alignment filter rejects candidate goals that fall behind the agent, and an agent for which no aligned goal can be found is removed from the scene to avoid training on degenerate configurations. The sampling bounds and filter thresholds are configurable, and we omit their values.

E Training Configuration

Table 14 lists the training hyperparameters shared by all experiments unless otherwise noted, and Table 15 summarizes where the two reported policies, the planner and the heterogeneous sim agent, differ. Simulation-level constants shared by both policies (episode length, decision clock, and observation budgets) are those of Table 7.

Table 14. Training hyperparameters. Values are shared by both reported policies; the settings on which the planner and the sim agent differ are collected in Table 15.

Category	Parameter	Value
<i>PPO</i>		
	Learning rate	5×10^{-4}
	Learning rate annealing	Linear decay
	Discount factor (γ)	0.99
	GAE λ	0.95
	PPO clip coefficient (ϵ)	0.2
	Value function clip coefficient	None (unclipped)
	Entropy coefficient	0.01
	Value function coefficient	0.5
	Max gradient norm	0.5
	Update epochs per rollout	2
<i>V-trace</i>		
	V-trace enabled	True
	$\bar{\rho}$ (IS ratio clip)	1.0
	$\bar{c}$ (trace coefficient clip)	1.0
<i>PopArt</i>		
	EMA decay (per step)	0.9997
	Minimum σ	10^{-4}
<i>Priority Sampling</i>		
	Priority exponent (α)	0.85
	IS correction β_0	0.85
	β annealing	Linear, to ≈ 0.978
<i>Optimizer</i>		
	Optimizer	Adam
	Adam β_1	0.9
	Adam β_2	0.999
	Adam ϵ	10^{-8}
<i>Architecture</i>		
	Num minibatches	16
	Total agents per GPU	512
	Environments per GPU	32
	Vectorized batch size	4

Table 15. Configuration of the two reported policies, grouped by topic. Shared optimization hyperparameters are those of Table 14; the reward and kinematic randomization ranges are detailed in Appendices B and D.4, and the rule-based road users in Section 4.4.

	Planner	Sim agent
<i>Policy</i>		
Architecture	Compact MLP, ~3.5M parameters	Heterogeneous MLP with per-type action heads, ~6.7M parameters
Controlled classes	Vehicles	Vehicles, pedestrians, cyclists
Action space	Discrete jerk grid	Per-type heads: jerk, unicycle, and compact bicycle
<i>Scenario</i>		
Initialization	Random on-lane placement	Random on-lane placement for all classes
Goal dropout	0.3	0.3
Traffic signals	Christmas controller	Christmas controller; pedestrians exempt from signal rules
Road users	Mixed vehicle, pedestrian, and obstacle generators	Vehicle generators only; pedestrians and cyclists are policy-controlled
<i>Domain randomization</i>		
Reward weights	Vehicle column of Table 8	All three type columns of Table 8
Kinematic coefficients	[0.5, 1.5] on all four	Vehicles [0.5, 1.5]; cyclists narrower ranges; pedestrians fixed
<i>Compute and data</i>		
Hardware	16 NVIDIA A100 80GB GPUs	32 NVIDIA A100 80GB GPUs
Map source	nuPlan	Waymo Open Motion Dataset